



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech



Software Engineering

2-JULY-2026

Contents

- Introduction to Software Engineering
- Software Development Life Cycle Models
- Software Requirements Analysis
- Software Design
- Object Oriented Design and UML Diagrams
- Software Testing

Introduction to Software Engineering



Introduction

- **Software** is more than **just a program code**. A program is an executable code, which serves some **computational purpose**.
- **Software** is considered to be **collection of executable programming code, associated libraries and documentations**. Software, when made for a specific requirement is called **software product**.
- **Engineering** on the other hand, is all about **developing products, using well-defined, scientific principles and methods**.

Importance of Software Engineering

- **Improves Quality:** Software engineering ensures the development of **reliable, error-free, and high-performance software**.
- **Reduces Cost and Time:** It helps minimize development time and cost through **proper planning and efficient** processes.
- **Handles Complexity:** It breaks down **complex systems into manageable parts**, making development easier and more organized.
- **Enhances Maintainability:** Well-engineered software is easier to **update, fix, and improve** in the future.
- **Boosts Customer Satisfaction:** It **delivers software** that meets user needs and performs as expected.

Characteristics of Good Software

The factors are divided into **three** categories:

1. Operational Phase : The phase where software is actively used in the real environment.

Characteristics:

- Delivers core functionality.
- Handles real-time user input/output.
- Must be stable and reliable.

2. Transitional Phase : The phase of moving software from development to production.

Characteristics:

- Includes installation & deployment.
- Involves user training and documentation.
- Requires environment setup and testing.

Characteristics of Good Software

3. Maintenance Phase : The phase after deployment for updates and issue resolution.

Characteristics:

- Fixes bugs and errors (corrective).
- Adapts to new environments (adaptive).
- Adds improvements (perfective) or prevents issues (preventive).

Software Development Life Cycle Models



Introduction

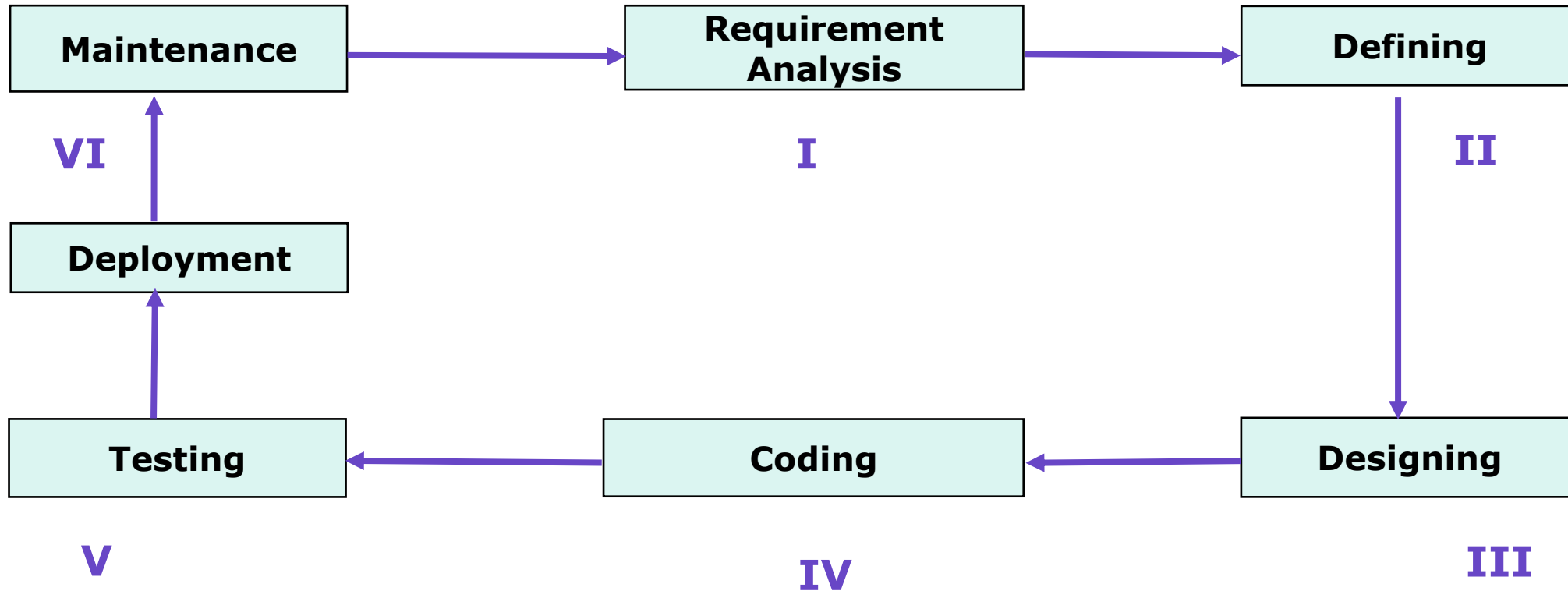
- **SDLC** is a **methodical approach to developing software** that ensures its **quality and correctness**.
- The SDLC process is designed to provide **high-quality software** that satisfies **customer requirements** and ensures system development is completed **within the schedule and budget constraints**.
- SDLC is a detailed plan that describes how to **plan, construct, and maintain specific software**.
- An **SDLC Model** is a **diagrammatic and graphical depiction of the software life cycle**.

Phases of SDLC

- **Seven phases** together constitute an **SDLC model**.
- These **phases** cover **every stage** in the **lifecycle of a software system**, starting from **requirement collection** and going all the way to **deployment and maintenance**.
- Each **SDLC life cycle phase** has its **own set of processes and deliverables** that feed into the next.
- The **seven phases of SDLC** include **planning, analysis, design, development, testing, implementation, and maintenance**.

Software Development Life Cycle Models

Phases of SDLC



Phase 1 - Requirement Collection

- The most **crucial and fundamental stage** in SDLC is **requirement collection**.
- It is carried out by **senior members of the team** with input from the **client or customer, the sales department, market surveys, and industry domain specialists**.
- This **data** is then used to **plan the basic project approach** and **conduct product feasibility studies on various aspects** described in the next phase of the SDLC methodology.
- The **requirement collection phase** is essential for **understanding the software product's business use case**.
- The **business analyst** sets up a **meeting with the client** to acquire all of the facts, such as **what the customer wants to construct, who will be the end user, and what purpose the product will serve**.

Phase 2 - Defining requirements and Feasibility Study

- Following the **completion of the requirement collection**, the next stage is to explicitly **describe and record the product needs and obtain approval from the client or market analysts**.
- This is accomplished through the **SRS (Software Requirement Specification) document**, which includes **all the product requirements to be defined and developed throughout the project life cycle**.

Phase 2 - Defining requirements and Feasibility Study

Once the SRS document is prepared, the requirements are evaluated for feasibility checks over these parameters:

- **Economical:** Can we finish the project within the budget?
- **Legal:** Can we handle this project in light of cyber law and other regulatory frameworks/compliances?
- **Operation feasibility:** Can we construct the activities that the client expects?
- **Technical:** Determine whether the current computer system is capable of supporting the software.
- **Schedule:** Determine whether or not the project can be completed within the time frame specified.

Phase 3 – Software Design

- In this phase, the **SRS document** is used as input to prepare the **DDS - Design Document Specification**.
- **Software architects** use the **requirements defined in SRS** as a reference to come up with the best architecture for the product being developed.
- Ideally, **multiple design approaches are proposed and documented in the DDS**.
- All essential stakeholders review this DDS, and the **optimal design strategy** for the product is chosen **based on many characteristics** such as **risk assessment, product robustness, design modularity, budget, and time restrictions**.
- A **design approach explicitly outlines** all of the **product's architectural components**, as well as its **communication and data flow representation** with **external and third-party modules** (if any).

Phase 4 – Coding

- In this phase, the **developer receives the design document**, and the **software design is converted into source code** in the most appropriate **high-level programming language**.
- It can be called the **most prolonged phase of SDLC**.

Phase 5 – Software Testing

- Once the coding phase is over and the **modules are ready for testing**, they are **deployed in the testing environment** and **tested against the requirements** to ensure that the **products address the needs identified and obtained** during the **requirements stage**.
- **Unit testing, integration testing, system testing, and acceptability testing** are all **performed during this stage**.
- The **QA (Quality Assurance) and testing teams** may discover **faults or defects**, which they report to developers.

Phase 5 – Software Testing

- The **development team fixes the bug** and sends it back to QA for another round of testing.
- This **procedure is repeated (retesting and regression testing) until the program is bug-free, stable, and meets the system's business requirements.**
- **Testers** refer to the **SRS document** to ensure that the **software meets the customer's standard.**

Phase 6 – Installation/Deployment

- Once the testing phase is over and no bugs or errors are left in the system, the deployment process starts.
- Through developer efforts, various modules or designs will be integrated into the core source code, typically by employing training environments to find further faults or defects.
- Finally, the **information system will be integrated into its environment.**
- After clearing this stage, the **program is theoretically market-ready** and can be **distributed to any end user.**

Phase 7 – Maintenance

- Once the built software is deployed, issues might arise while the code runs in the production environment and need to be solved on the go.
- In SDLC models, this is called the **maintenance phase of a software project**, and it consists of **mainly three activities-**
- **Bug fixing** - bugs or errors arising due to unforeseen circumstances which were not taken care of during testing.
- **Upgrading** to newer versions
- **Enhancement** - adding new features
- The focus of this phase is to ensure that needs continue to be met and that the system continues to perform as per the requirements collected in the first phase.

Types of SDLC Models



Types of SDLC Models

SDLC Model

- Various SDLC models have been developed to manage **software projects efficiently**, each suited to different **project needs** and team dynamics. Below are **widely used SDLC models**, each offering a unique approach to software development,

1. Waterfall Model

2. Iterative Model

3. V Model

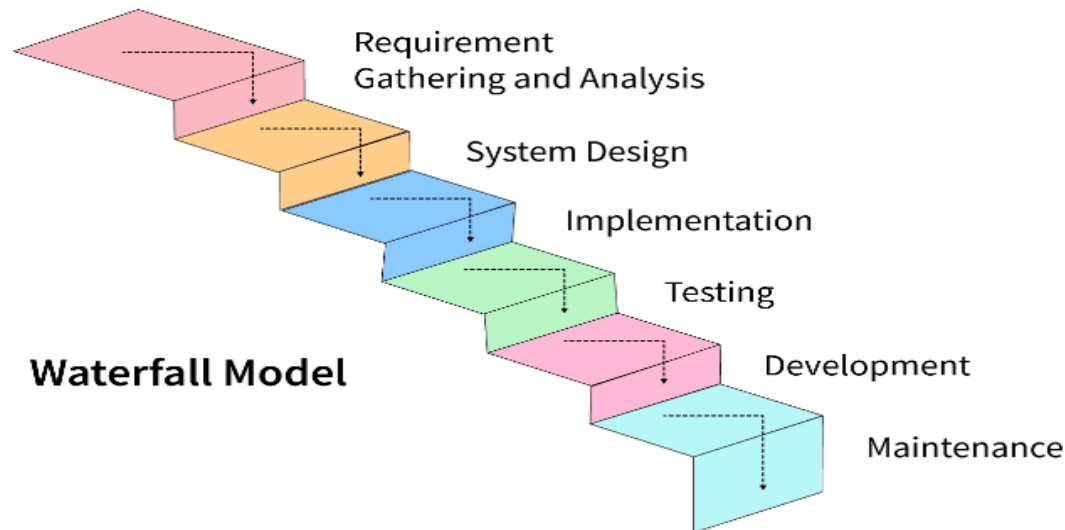
4. Spiral Model

5. Agile Model

Types of SDLC Models

Waterfall Model

- The Waterfall Model is one of the **earliest and most traditional software development life cycle** (SDLC) models. It is **a linear and sequential approach**, where each phase must be **completed** before moving on to the next. The process **flows steadily downwards**, like a waterfall, through several distinct phases.



Waterfall Model : Advantages and Disadvantages

Advantages

- ❖ Easy to understand, easy to use
- ❖ Provide structure to inexperienced staff
- ❖ Milestones are well understood
- ❖ Sets requirements stability
- ❖ Good for management control (plan, staff, track)
- ❖ Works well when quality is more important than cost or schedule

Disadvantages

- ❖ All requirements must be known upfront
- ❖ Deliverables created for each phase are considered frozen – inhibits flexibility
- ❖ Can give a false impression of progress
- ❖ Does not reflect problem-solving nature of software development – iterations of phases
- ❖ Integration is one big bang at the end
- ❖ Little opportunity for customer to preview the system (until it may be too late)

Types of SDLC Models

When to use waterfall model?

- Requirements are very well known
- Product definition is stable
- Technology is understood
- New version of an existing product
- Porting an existing product to a new platform



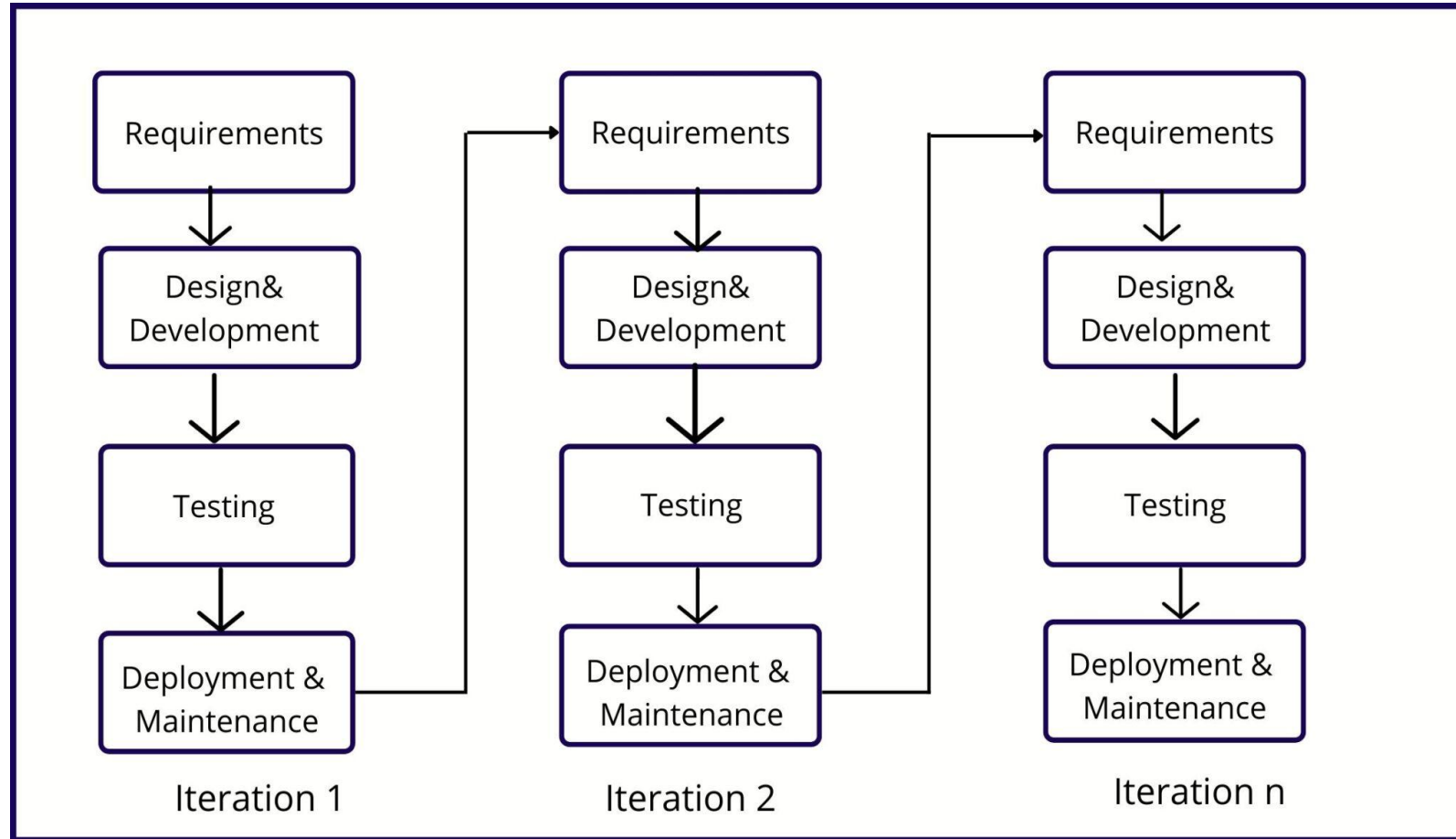
Types of SDLC Models

Iterative Model

- The **iterative process model** is the implementation of the software development life cycle in which the **initial development is started based on the initial requirements** and **more features** are added to the base software product with the **ongoing iterations until the final system is created**.
- Iterative development, in a nutshell, is a method of **breaking down the software development of a massive program into smaller components**.

Types of SDLC Models

Iterative Model



Iterative Model : Advantages and Disadvantages

Advantages

- ❖ Produces a working software much quickly and early during the SDLC.
- ❖ Very flexible. As new functionality can be added to it at any time of development.
- ❖ is less costly to change requirements as compared to the other process models.
- ❖ The end-user or the stakeholders can give their feedback quickly.
- ❖ The errors and bugs in the system can be identified early.
- ❖ Takes smaller development teams as compared to other process models.

Disadvantages

- ❖ Problems pertaining to the system architecture can come up because all the requirements are not gathered upfront.
- ❖ It is not a good choice for small projects.
- ❖ More resource-intensive than waterfall model.
- ❖ Risk analysis requires highly qualified specialists to check the risks in our system.
- ❖ The whole process is difficult to manage.

Types of SDLC Models

When to use Iterative model?

- The iterative model really starts to shine when its in the **hands of a smaller, more agile team.**
- The iterative model is really beneficial as it **can accommodate changes.**
- **Early work product is needed.**

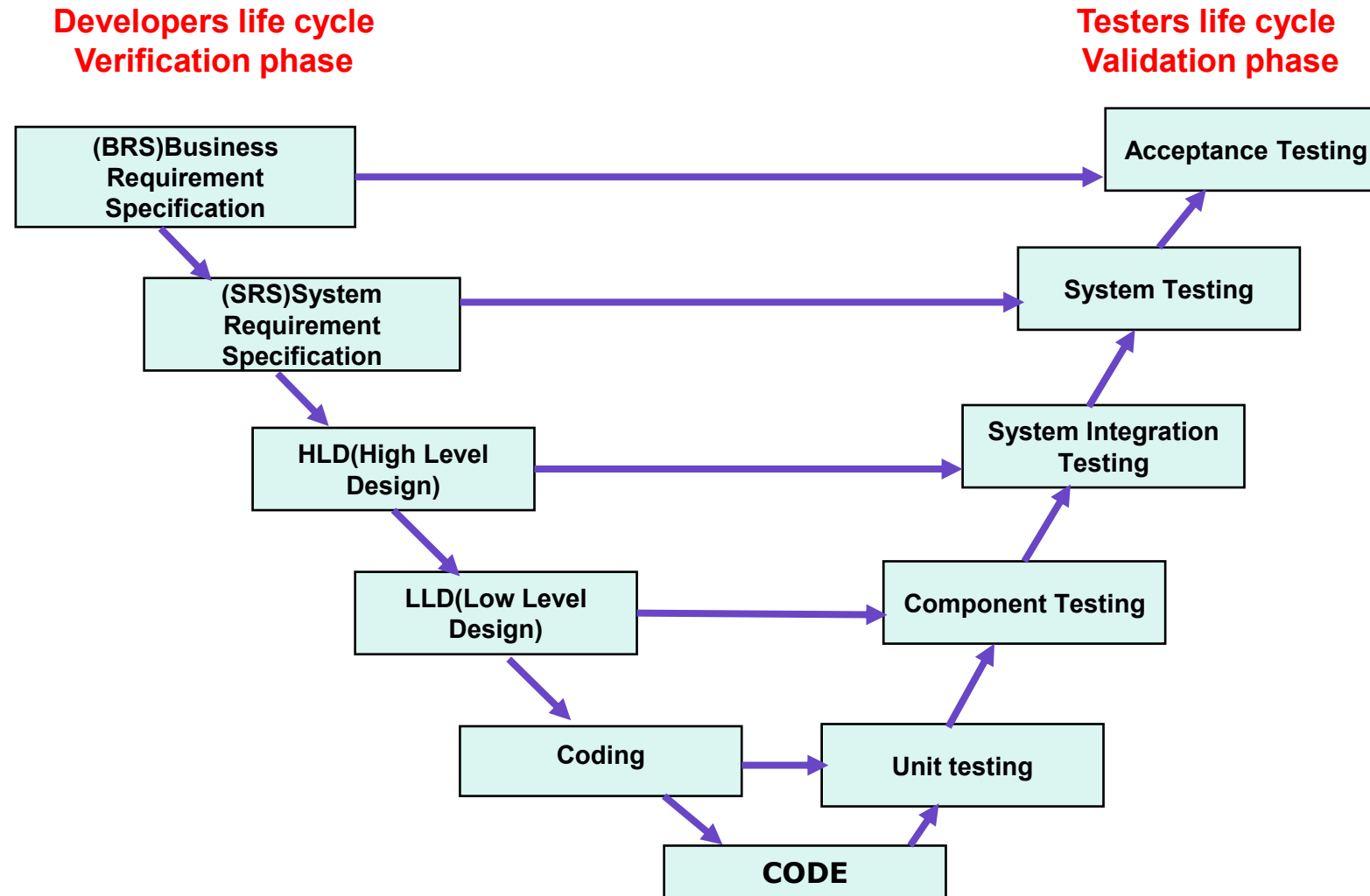
Types of SDLC Models

V model - Verification and Validation model

- **V model** is a software development lifecycle model (SDLC) in which each step executes in a **sequential manner with parallel testing for each development stage**.
- The model is known as the V model because the diagram of the V model is quite similar to the V shape.
- V model is an extension of the Waterfall model, with the improvement that **each development phase will go through a testing phase before moving forward**.
- V model is a **strict model** as the development **only moves forward when the previous step is completed**, and this is made sure by doing testing after each development activity.

Types of SDLC Models

V Model



V Model : Advantages and Disadvantages

Advantages

- ❖ Emphasize planning for verification and validation of the product in early stages of product development
- ❖ Each deliverable must be testable
- ❖ Project management can track progress by milestones
- ❖ Easy to use

Disadvantages

- ❖ Does not easily handle concurrent events
- ❖ Does not handle iterations or phases
- ❖ Does not easily handle dynamic changes in requirements
- ❖ Does not contain risk analysis activities

Types of SDLC Models

When to use V model?

- Excellent choice for **systems requiring high reliability** – Eg. **hospital patient control applications, Nuclear Power Stations, Weather Monitoring Etc.,**
- **All requirements are known up-front**
- When it can be modified to **handle changing requirements beyond analysis phase**
- **Solution and technology are known**

Types of SDLC Models

Spiral Model

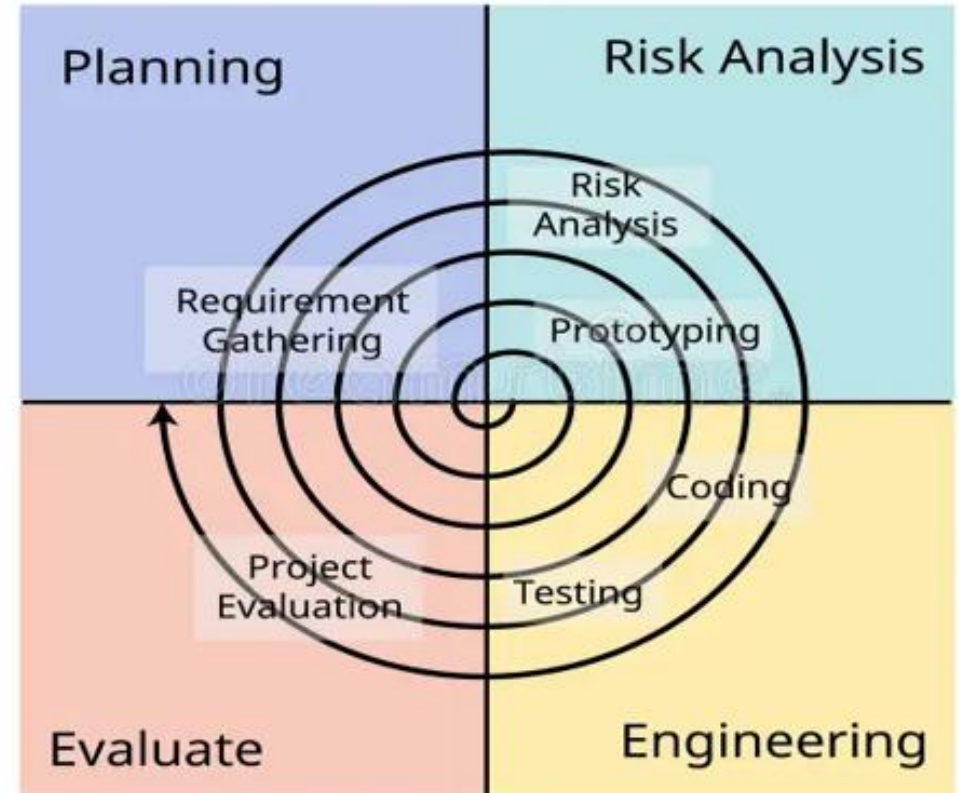
- The Spiral Model is a risk-driven software development process model that combines elements of both **iterative development and the Waterfall model**.
- It is particularly effective for **large, complex, and high-risk projects**.
- This model organizes the development process into a spiral with **four major phases** repeated in each loop
- Each loop of the spiral represents a phase of the software process and includes activities like **prototyping, risk handling, and customer evaluation**.

Types of SDLC Models

Spiral Model

Four major phases repeated in each loop:

- 1. Planning** – Define objectives, identify resources, and gather requirements.
- 2. Risk Analysis** – Evaluate alternatives, identify and mitigate risks.
- 3. Engineering** – Perform design, coding, and testing.
- 4. Evaluation** – Assess the project and decide the next iteration.



Spiral Model : Advantages and Disadvantages

Advantages

- ❖ Early identification and handling of risks
- ❖ Continuous customer involvement improves accuracy
- ❖ Easily accommodates changes during development
- ❖ Prototyping helps clarify requirements early
- ❖ Well-suited for large and high-risk projects

Disadvantages

- ❖ Not suitable for small or low-budget projects
- ❖ Complex to manage and plan effectively
- ❖ Can be time-consuming with repeated iterations
- ❖ Risk of uncontrolled scope expansion
- ❖ Requires experienced developers and risk analysts

Types of SDLC Models

When to use Spiral model?

- Ideal for large, complex, and high-risk projects where **risk management is critical**.
- When the requirements are **unclear or expected to change** frequently during development.
- Suitable when the client needs to **see prototypes** for better understanding and feedback.
- Works best when the development team has experience in **risk analysis** and managing iterative processes.

Types of SDLC Models

Agile Model

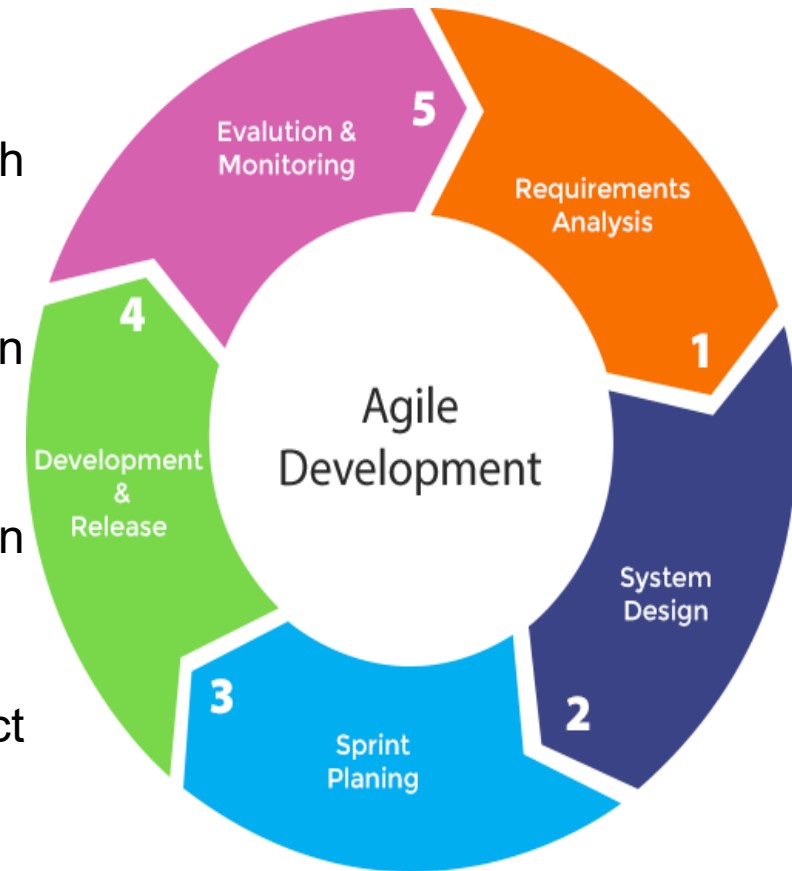
- The Agile Model is a **modern approach to software development** that emphasizes flexibility, collaboration, and customer feedback.
- Unlike traditional linear models, Agile is **iterative and incremental**, allowing teams to deliver working software in **small, manageable parts called sprints**.
- Agile promotes **adaptive planning**, continuous **improvement**, and **quick responses** to change, making it ideal for **dynamic and fast-paced projects**.
- It encourages **close interaction** between **developers, testers, and customers** throughout the development process.

Types of SDLC Models

Agile Model

Agile Development Phases

- **Requirements Analysis** – Gather and define user needs through stakeholder collaboration.
- **System Design** – Plan the system architecture and break down features into components.
- **Sprint Planning** – Select tasks for the sprint and assign responsibilities within the team.
- **Development & Release** – Build, test, and deliver a working product increment.
- **Evaluation & Monitoring** – Review progress, collect feedback, and improve in the next sprint.



Types of SDLC Models

Agile Model

- The **entire project** is divided into **smaller portions or sprints** with Agile to **reduce project delivery time and hazards**.
- An iteration requires a team to go through the entire software development cycle.
- Within a **single iteration**, an **Agile team** will
 - **map out the requirements**
 - **develop the user stories**
 - **test their software**
 - **produce an end deliverable**
 - **request for user feedback.**

Types of SDLC Models

When to use agile model?

- **Requirements** are evolving – when client needs or goals may change frequently.
- **Frequent updates** are needed – when continuous delivery and feedback are essential.
- **Close collaboration** is possible – when customers and developers can interact regularly.
- **Rapid and flexible development** – when early and incremental delivery is important.

Types of SDLC Models

Comparison of SDLC Models

Feature / Model	Waterfall	Iterative	V-Model	Spiral	Agile
Process Flow	Linear, sequential	Cyclic, repeated phases	Linear + parallel testing	Cyclic with risk analysis	Iterative & incremental
Flexibility	Low	Some changes allowed	Very low	High	Very flexible
Risk Handling	Poor	Moderate	Poor	Excellent	Good
Customer Involvement	Only at start & end	Moderate	Only at start & end	High	Very high
Cost of Changes	High	Moderate	High	Low to moderate	Low
Best Suited For	Simple, fixed projects	Evolving projects	Critical systems	High-risk, large projects	Dynamic, fast-changing needs

Types of SDLC Models

How to choose a SDLC model ?

- Based on how **clearly requirements** are defined
- Depends on **project size and duration**
- Depends on **project complexity**
- According to the **available budget**
- Based on the **level of documentation** required by the client

Software Requirements Analysis



Introduction

- Every **new software product or service developed** in any corporation is in **response to a business requirement**.
- Despite spending significant time and money on the development, there may be a **mismatch between the needed product** (according to the business requirement) **and the final result**.
- Hence, to **avoid severe problems in the future**, a focused and **complete requirements analysis is required in the early stages of every project**.

What is Software Requirement?

- A **software requirement** is a **capability** that the user requires to solve an issue or achieve a goal.
- In other words, a **requirement** is a **software capability** that a **system or system module** must meet or possess to **satisfy a contract, standard, or specification**.
- Ultimately, we want to provide high-quality software that matches clients' actual needs on schedule and within budget.

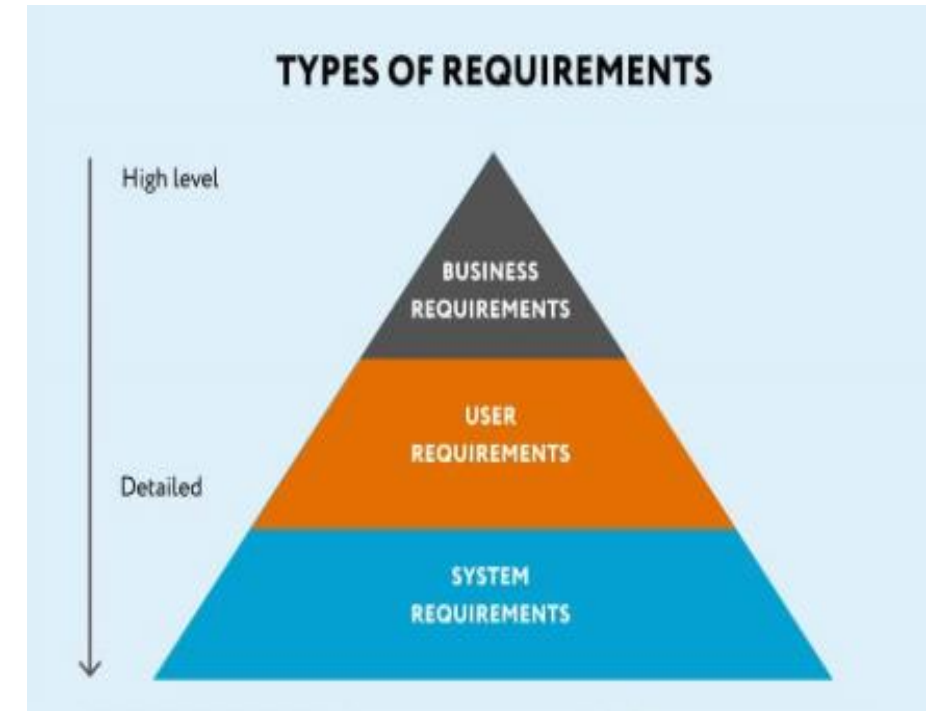
What is Software Requirement?

- All **project stakeholders - developers, end users, software managers, and customer managers - must agree on what the product will look like**, or someone will be surprised **when it is delivered**, which is an undesirable event.
- Therefore, while establishing the requirements for a software product, we need methods to accurately **capture, interpret, and express** the voice of the customer.

Software Requirements Analysis

Types of Requirements

- In a Software Requirements Specification (**SRS**), requirements are categorized to ensure clarity, traceability, and alignment with both **business goals** and **user needs**.
- The main types of requirements include **Business Requirements**, **User Requirements**, and **System Requirements**.



Types of Requirements - Business requirements

- **Business requirements** describe the **high-level business needs**, such as carving a market share, reducing customer churn, or improving the customers' lifetime value.
- **Example:**
 - The hotel management system must provide an online booking platform that allows guests to reserve rooms, view room availability, and make payments online.

Types of Requirements - User requirements

- **User requirements** cover the **different goals your users can achieve using the product** and are commonly documented in the form of user stories, use cases, and scenarios.
- **Example: User Requirements for Hotel Management System.**
- **User Authentication and Authorization:**
 - Users (hotel staff) should be able to log in to the system using their username and password.
 - Different user roles should be defined (e.g., receptionist, manager), each with specific access rights and permissions.

Types of Requirements - User requirements

User Story

- A **user story focuses on the experience, what the user wants to achieve with the product.**
- **Requirements**, on the other hand, **focus on functionality – what the product should do.**
- Requirements are developed at a later stage. During a **sprint planning meeting, the team usually decides what stories to tackle.**
- They then move on to discuss the requirements and functionality that each user story requires.
- Once agreed upon, these requirements are added to the story or incorporated into a separate product requirements document (PRD).

Types of Requirements - System requirements

- **System or Product requirements** describe how the system needs to operate to meet the business and user requirements. They include **functional requirements** and **non-functional requirements**.
- **Example: Product Requirements for Hotel Management System - User Interface (UI) Design:**
 - The system should have an intuitive and user-friendly interface, with easy navigation and clear visual elements.
 - UI elements should be responsive and adaptable to different screen sizes and resolutions.

System Requirements - Functional requirements

- **Functional requirements** are product features that **developers must implement** to enable the users to achieve their goals. They **define the basic system behavior** under specific conditions.
- **Functional requirements examples:**
 - The system must send a **confirmation email** whenever an order is placed.
 - The system must allow **blog visitors** to sign up for the newsletter by leaving their email.
 - The system must allow users to **verify their accounts** using their phone number.

System Requirements - Functional Requirements

- But **stories can be a useful tool for deriving requirements with the user in mind.**

For example:

- **User story:** As an existing user, I want to be able to log into my account.

Functional requirements

- The system must allow users to **log into their account** by entering their email and password.
- The system must allow users to log in with their **Google accounts**.
- The system must allow users to **reset their password** by clicking on "I forgot my password" and receiving a link to their verified email address

System Requirements - Non-functional Requirements

- Non-functional requirements place constraints on **how the product should do it**. They can be expressed in the following form:

Functional requirement: "The system must do [requirement]."

Non-functional requirement: "The system shall be [requirement]."

- Non-functional requirements (also known as **"quality requirements"** or **"quality attributes"**) are more abstract.
- They impose constraints on the implementation of the functional requirements in terms of **performance, security, reliability, scalability, portability, and so on.**

Restaurant Application - Example

1. Major Requirements : The key features the application **must support** to run a restaurant business digitally.

Feature	Explanation
Online Food Menu	Customers can view all dishes with images, prices, and descriptions.
Table Reservation	Users can book tables in advance for a specific date and time.
Online Ordering	Customers can order food online and customize items (e.g., extra cheese).
Real-Time Order Tracking	Users can see the current status of their orders.
Payment Integration	Secure online payment options (cards, wallets, UPI).
Admin Dashboard	Admin can manage menu, orders, customers, and feedback.
Customer Feedback	Users can rate and review food and services.
Notifications	Alerts via SMS/email/app about order status or reservations.

Restaurant Application - Example

2. Business Rules : Fixed rules or conditions that define how the system should behave.

Rule	Explanation
Table Booking Availability	Only allow reservations for free tables.
Registration Required	Users must sign up to order or reserve.
Order Cancellation Policy	Cancellation only allowed before food preparation starts.
One Coupon per Order	Apply only one discount code per purchase.
Delivery Radius Limit	Deliver only to addresses within a set distance from the restaurant.
Online Orders are Prepaid	Payment must be made while placing the order.

Restaurant Application - Example

3. Functional Requirements : What the system **should do** (actions, features, workflows).

Module	Explanation
User	Users can sign up, log in, update profiles, and recover passwords.
Menu & Ordering	Users can see the menu, add items to cart, place orders, and track history.
Reservation	Book tables by selecting date, time, and number of guests.
Payments	Use digital payment methods and apply valid discounts.
Order Tracking	Show live order progress (like “Preparing”, “Out for delivery”).
Admin Panel	Admin can add new dishes, manage user orders, and see customer feedback.

Restaurant Application - Example

4. Non-Functional Requirements : Describe **how well** the system should perform.

Quality Attribute	Explanation
Performance	The app should handle many users at the same time without slowing down.
Availability	The app should be available nearly all the time (e.g., 99.9% uptime).
Security	User info and payments must be protected with encryption and secure logins.
Usability	Easy-to-use interface, simple navigation, and mobile-friendly design.
Scalability	App should support more restaurants or branches in the future.
Maintainability	Code should be well-organized and easy to fix or update.
Portability	Should work well on all modern browsers and mobile devices.

Restaurant Application - Example

5. Validation Rules : Specific checks to ensure input and actions are valid.

Validation Rule	Explanation
Email format check	Must be a valid email (e.g., user@example.com).
Password strength	Must contain upper & lowercase letters, digits, and be at least 8 characters long.
Reservation date check	Cannot book a past date/time.
Mobile number format	Must be a valid 10-digit number.
Required fields	Name, address, and payment info must not be left empty.
Coupon code validity	Check if the coupon is active and applicable to current cart items.
Order limit	Maximum number of items per order (e.g., 20 items).
Delivery location validation	Address must fall within serviceable PIN codes.

Software Design



Introduction

- Software Design is the process of **planning and defining** how a software system **will work and how its components will interact**. It involves creating a **blueprint for the system** before actual coding begins.

It focuses on:

- Breaking down the system into **modules or components**
- **Defining** the structure, data flow, and interfaces
- Ensuring the software is **reliable, efficient, and maintainable**
- In short, software design **turns requirements into a clear technical plan** for developers to follow.

Levels of Software Design

The **three most important levels of software design** are:

1. Architectural Design

- Defines the **overall system structure**, major components, and their interactions.

2. High-Level Design (also called System Design)

- Breaks the system into **modules** and defines their **functions** and data flow.

3. Detailed Design (also called Low-Level Design)

- Specifies the **internal logic** of each module, including algorithms and data structures.

Software Design

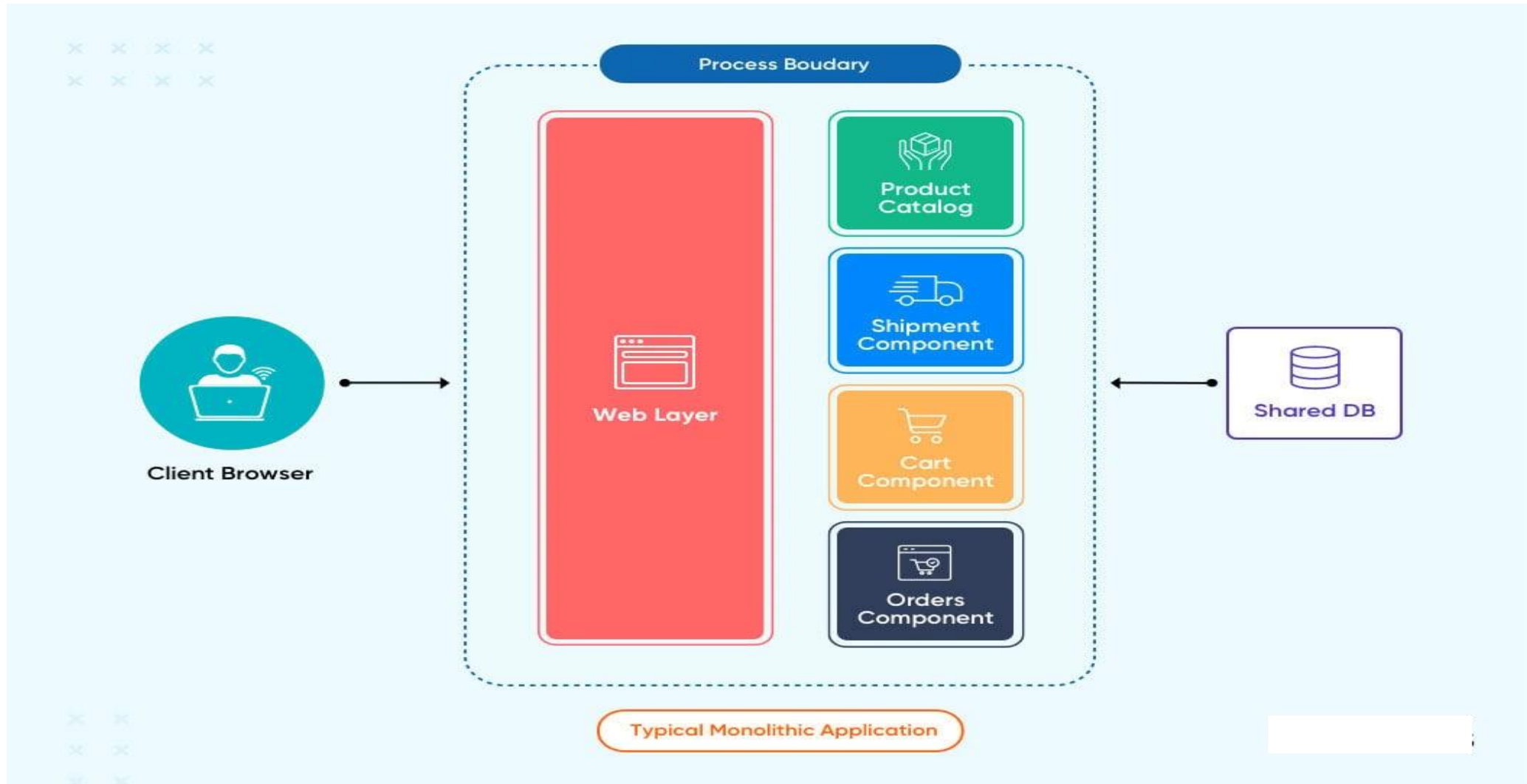
Architectural Design

- Making important choices about system organization, such as **choosing appropriate frameworks, patterns, and technologies to guarantee performance, maintainability, and dependability**, is essential.
- A good software architecture design helps define **performance, maintainability, quality, and scalability**.
- The goal is to ensure that your **software is flexible enough to adapt**, as **new requirements emerge**.

Types of Architectural Design

Architecture Type	Definition	Benefits	Drawbacks
Monolithic Architecture	• Each component of an application is deployed as a single, self-contained unit and is closely integrated, following a traditional approach to software design. • The user interface, business logic, and data access layer are all developed and deployed as a single codebase and executable	• Simple and easy to understand • Easy deployment • Improved performance	• Scalability limitations • Maintenance complexity • Vendor lock-in

Monolithic Architecture – Online Shopping System



Types of Architectural Design

Architecture Type	Definition	Benefits	Drawbacks
Microservices Architecture	• An application built using the microservices architectural style is made up of independently deployable, loosely coupled services arranged according to business functions. • Unlike monolithic architectures, microservices divide the application into smaller, more focused services, each in charge of a particular feature or function.	• Scalability • Flexibility • Maintainability	• Complexity • Debugging challenges • Increased infrastructure costs

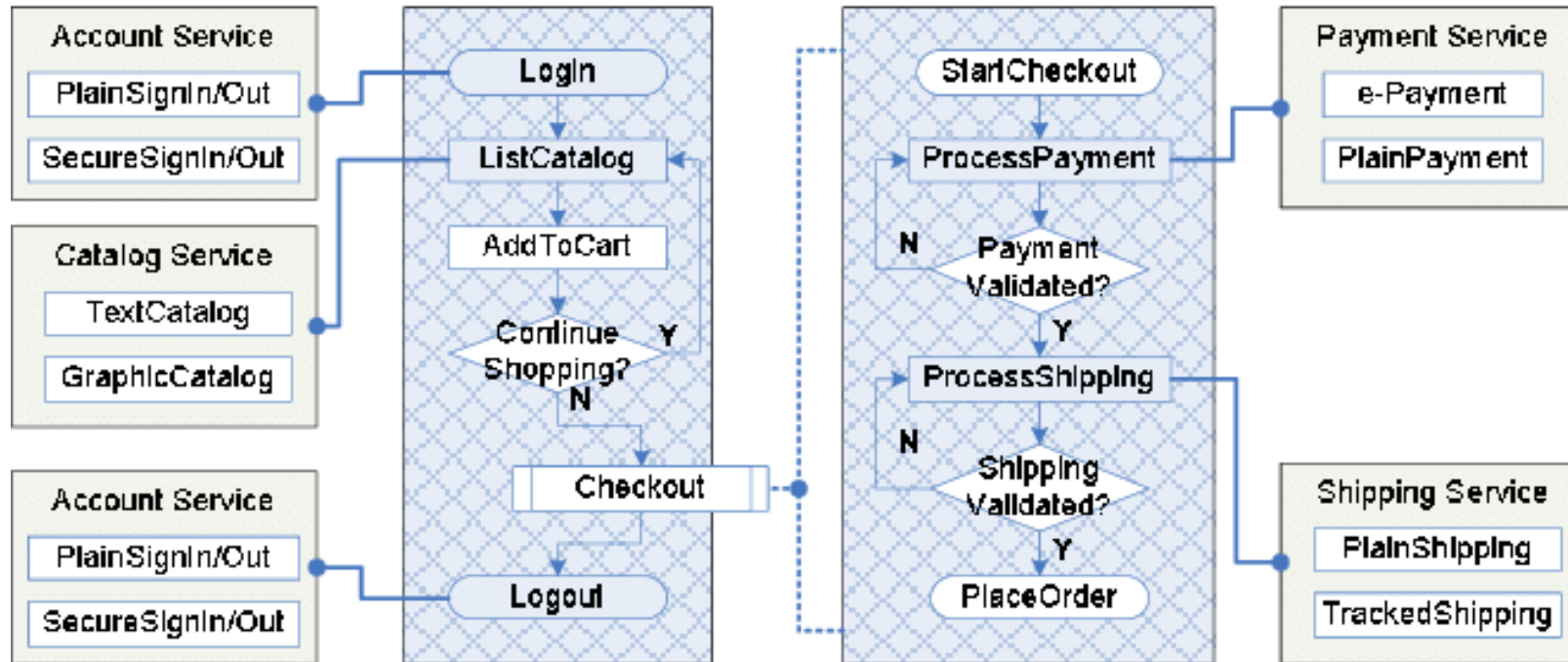
Microservices Architecture – Online Shopping System



Types of Architectural Design

Architecture Type	Definition	Benefits	Drawbacks
Service-Oriented Architecture (SOA)	• It is a design style that focuses on autonomous, reusable services that communicate with each other to deliver functionalities. • SOA emphasizes loose coupling and standardized communication protocols, allowing services to be developed, deployed, and scaled independently.	• Loose coupling • Reusability • Flexibility	• Complexity • Performance overhead • Steeper learning curve

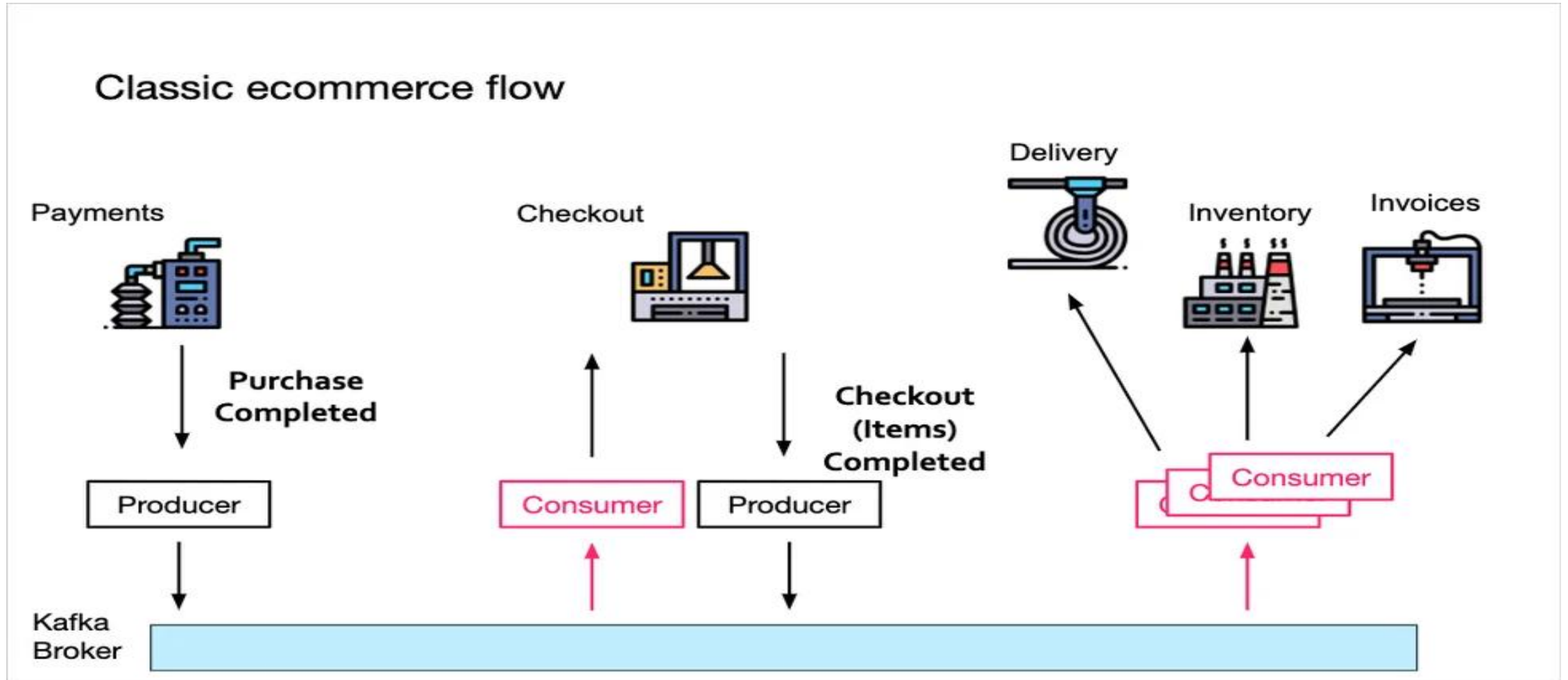
Service Oriented Architecture – Online Shopping System



Types of Architectural Design

Architecture Type	Definition	Benefits	Drawbacks
Event-Driven Architecture (EDA)	• Components interact and react to events that take place within or between systems. • EDA propagates events asynchronously to interested components or services. • Events can be generated by various sources, including user actions, system state changes, or external triggers.	• Loose coupling • Scalability • Real-time processing	• Complexity • Ordering complexities • State management

Event-Driven Architecture – Online Shopping System



Types of Architectural Design

Architecture Type	Definition	Benefits	Drawbacks
Layered Architecture	• Sometimes called n-tier architecture, divide an application into several layers or tiers, each in charge of a particular functional area. • An application is separated into horizontal layers in a layered architecture, where each layer denotes a different level of abstraction and concern separation.	• Separation of concerns • Maintainability • Reusability	• Tight coupling • Performance overhead • Limited flexibility

Layered Architecture – Online Shopping System

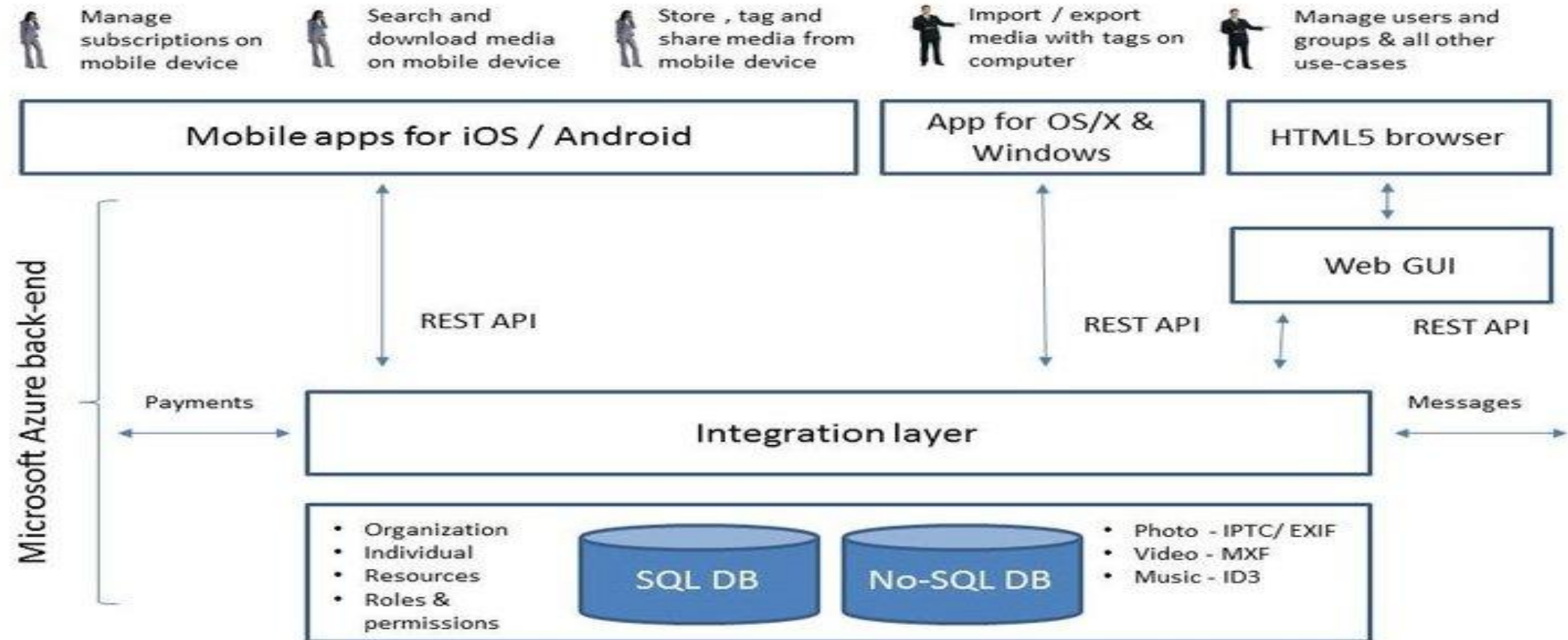


High-Level Design

- The high-level design **deconstructs the architectural design's** 'single entity-multiple component' notion into a **less abstract perspective of subsystems and modules**, depicting their interaction with one another.
- High-level design is concerned with **how the system and its components can be implemented as modules**.
- It recognizes the **modular structure of each subsystem**, as well as their relationship and interaction with one another.

Software Design

High-Level Design



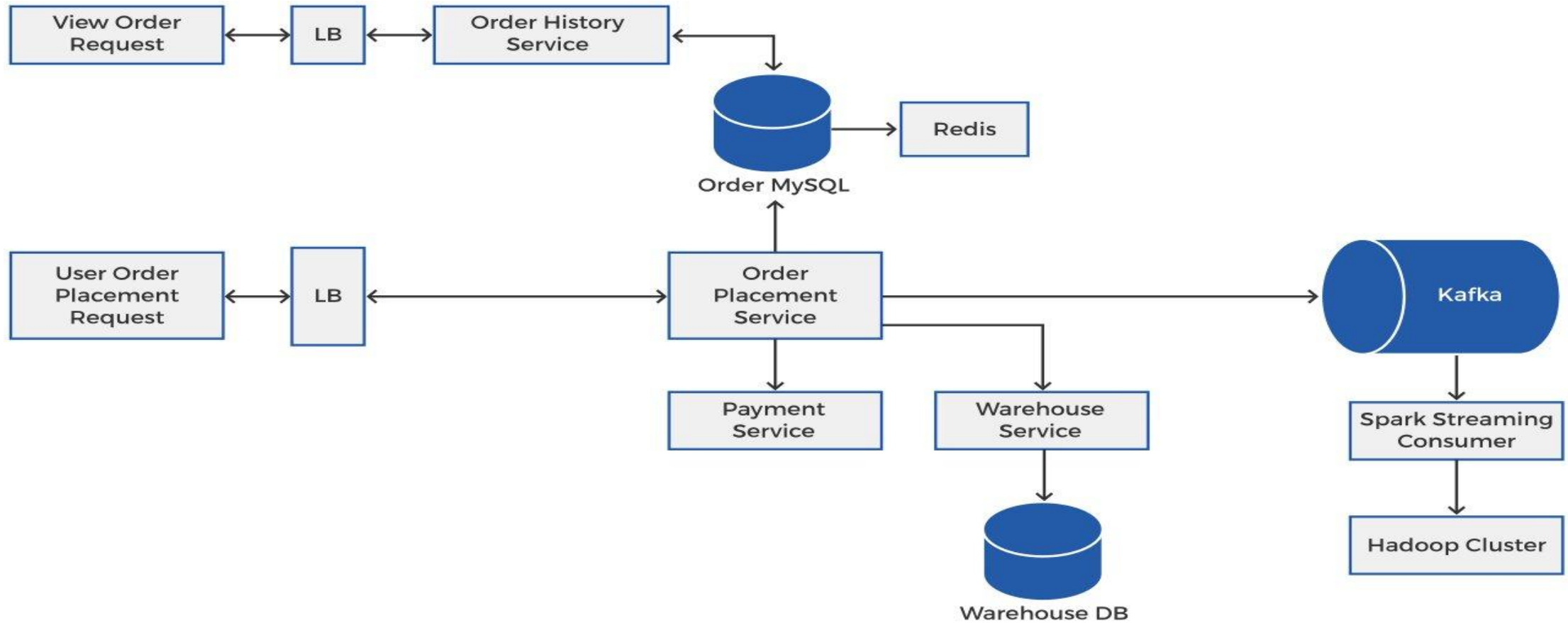
Detailed Design or Low Level Design

After the high-level design is completed, the detailed design begins.

- Each module is extensively investigated at this level of **software design to establish the data structures and algorithms to be used.**
- Finally, a module specification document is used to document the stage's outcome.
- It defines the **logical structure of each module as well as its interfaces with other modules.**

Software Design

Low Level Design - Online Shopping System (Order Request Module)



Comparison

Level	Description	Focus Area	Output
Architectural Design	Defines the overall structure and major components of the system	System architecture & component interaction	Architecture diagram, component relationships
High-Level Design	Breaks the system into modules and defines their functionality and data flow	Module structure & communication	Module specifications, data flow diagrams
Detailed Design	Specifies the internal logic of each module, including algorithms and data structures	Internal working of modules	Pseudocode, flowcharts, class/interface designs

Object Oriented Design and UML Diagrams



Introduction

- Object-Oriented Analysis and Design (OOAD) is a methodology used in software engineering that focuses on designing a **system using objects**, which are instances of classes.
- OOAD combines both **analysis** (understanding and modeling what the system should do) and **design** (how the system will achieve it) using object-oriented concepts like **encapsulation, inheritance, abstraction, and polymorphism**.
- OOAD helps developers create systems that are more **modular, reusable, maintainable**, and easier to understand by aligning the software design with **real-world entities and their interactions**.

Stages of OOD

The object-oriented design process includes two main stages:

1. System design

- The **entire architecture of the intended system is designed** at this stage.
- The system is envisioned as a **collection of interacting subsystems, each comprising a hierarchy of interacting objects classified into classes.**
- The system analysis model and the proposed system architecture are used to design the system. The emphasis here is on the **system's objects rather than the system's processes.**

Stages of OOD

2. Object Design

- A design model is created in this phase based on the models created in the **system analysis phase** and the architecture **built in the system design phase**.
- All of the **requisite classes** have been **recognized**.
- The **relationships between the specified classes are established, and class hierarchies are identified**.
- In addition, the developer creates the internal details of the classes and their relationships, such as the **data structure for each attribute and the algorithms for the operations**.

Unified Modeling Language(UML)

- UML stands for **Unified Modeling Language**, it is a standardized way to **visualize, design, and document** software systems.

Why is UML used?

- Helps **understand system structure and behavior** before coding.
- Provides a **common language** for developers, designers, and stakeholders.
- Supports **object-oriented design** and system planning.

Real-World Applications:

- Used in **software design diagrams** (e.g., class, use-case, sequence).
- Applied in **banking systems, e-commerce platforms, and embedded systems**.
- Widely used in **agile, waterfall, and model-driven development**.

UML Diagrams

- **UML (Unified Modeling Language) diagrams** are visual tools used to model, design, and document software systems. They help represent the **structure** and **behavior** of a system in a clear and standardized way.

Where UML Diagrams Are Used

- **Software Design:** To plan system architecture, workflows, and interactions before coding.
- **Requirement Analysis:** To visualize user needs and system functionality (e.g., Use Case diagrams).
- **System Documentation:** To maintain clear, up-to-date design documents for teams.
- **Communication:** To help developers, testers, and stakeholders understand the system.
- **Project Planning:** To estimate scope, dependencies, and design complexity.

UML Diagrams

- **UML (Unified Modeling Language) diagrams** are visual tools used to model, design, and document software systems. They help represent the **structure** and **behavior** of a system in a clear and standardized way.

Where UML Diagrams Are Used

- **Software Design:** To plan system architecture, workflows, and interactions before coding.
- **Requirement Analysis:** To visualize user needs and system functionality (e.g., Use Case diagrams).
- **System Documentation:** To maintain clear, up-to-date design documents for teams.
- **Communication:** To help developers, testers, and stakeholders understand the system.
- **Project Planning:** To estimate scope, dependencies, and design complexity.

UML Diagrams

UML Building Blocks

- They are the **fundamental parts** used to **create** UML diagrams.
- They help **model the structure** and **behavior** of a software system clearly and consistently.

Main UML Building Blocks

1. Elements – The core components used in diagrams

- **Structural Elements:** Represent static parts (e.g., classes, interfaces, nodes).
- **Behavioral Elements:** Represent dynamic aspects (e.g., activities, use cases, interactions).
- **Grouping Elements:** Organize other elements (e.g., packages).
- **Annotational Elements:** Add explanatory notes or comments.

UML Diagrams

2. Relationships – Show how elements are connected

- **Association** : A **basic** relationship showing how **two classes** are connected or interact.
Example: A **Student** is associated with a **Course** because they enroll in it.
- **Generalization** : Represents inheritance between a more **general (parent)** and a more **specific (child)** class.
Example: A **Dog** class inherits from an **Animal** class because all dogs are animals.
- **Dependency** : One **class** uses **another temporarily**, meaning changes in one may affect the other.
Example: A **Printer** class depends on a **Document** class to print files.

UML Diagrams

- **Aggregation** : A "whole-part" relationship where the part can exist independently of the whole.

Example: A **Library** has **Books**, but the books can still exist without the library.

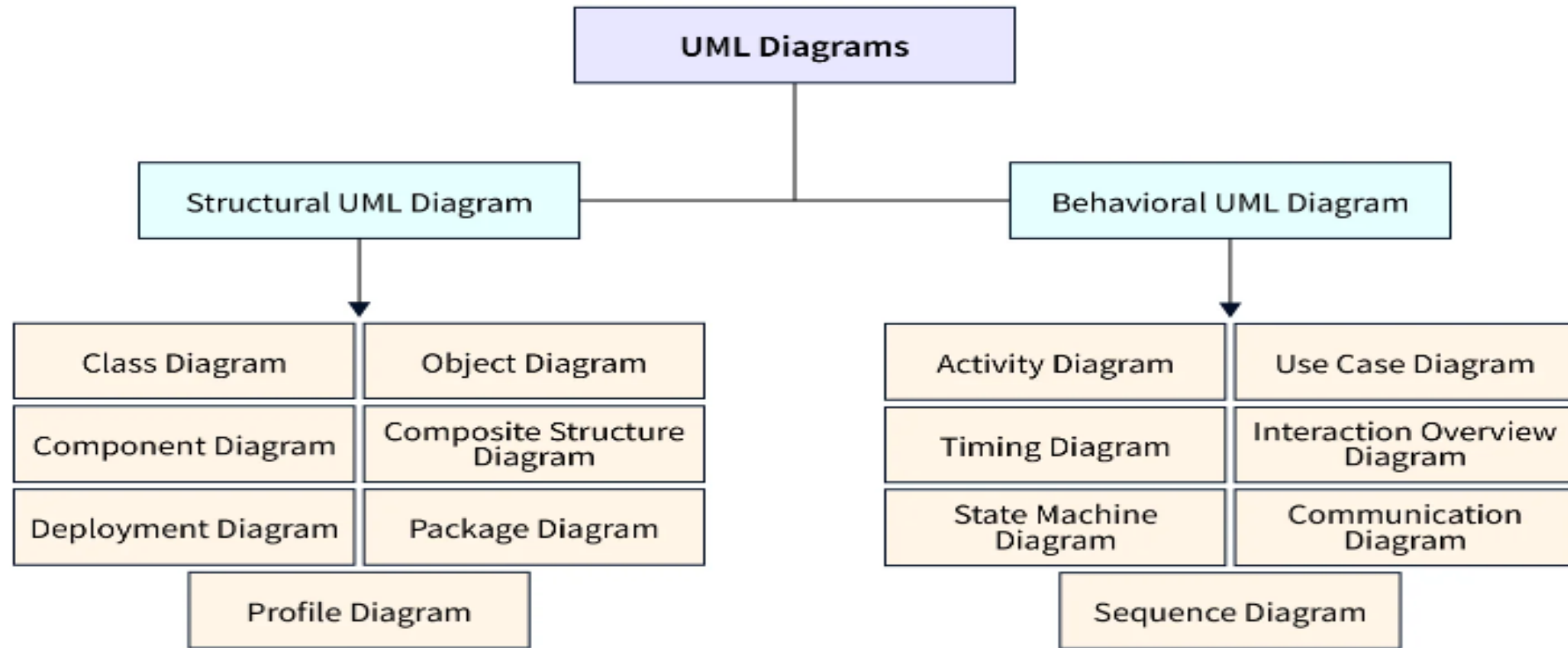
- **Composition** : A strong "whole-part" relationship where the part cannot exist without the whole.

Example: A **House** is made up of **Rooms**, and if the house is destroyed, the rooms cease to exist too.

Types of UML Diagrams

- UML provides **numerous sorts of diagrams** to depict the functioning of a system or program in **pictorial format**.
- These diagrams are classified into two groups: **structural diagrams and behavioral diagrams**.
 - The **structural diagram** represents the **static part of the system**.
 - The **behavioral diagram** depicts the system's **both static and dynamic aspects**.

Types of UML Diagrams



Structural Diagram

- **Structural diagrams** depict the **static structure of the elements in your system. i.e., how one object relates to another.**
- It shows the things in the system – **classes, objects, packages or modules, physical nodes, components, and interfaces.**
- For example, just as the static aspects of a house encompass the existence and placement of such things as walls, doors, windows, pipes, wires, and vents.

Structural Diagram

1. **Class Diagram** : Shows the **blueprint** of the system with classes, attributes, methods, and relationships. It is essential for designing and understanding object-oriented systems.
2. **Object Diagram** : Displays a **snapshot of instances (objects)** and their **relationships** at a particular moment. Helps visualize object interactions during system runtime.
3. **Component Diagram** : Illustrates how **software components** (modules, libraries) are **organized and interact**. Useful for understanding and designing system architecture.

Structural Diagram

4. Deployment Diagram : Shows the **physical arrangement of hardware** and how **software** is deployed on it. Helpful in mapping system software onto actual hardware components.

5. Package Diagram : Organizes **classes and elements into packages**, showing their **dependencies**. Makes managing **large-scale systems** easier by grouping related components.

6. Composite Structure Diagram : Reveals the **internal structure of a class** or component and how its parts interact. Useful for modeling the inner workings of complex elements.

Structural Diagram

7. Profile Diagram : Used to **customize UML** for specific domains using stereotypes and constraints.

Helps extend standard UML to meet **specialized modeling needs**.

Behavioral Diagram

- **Behavioral diagrams** are used to visualize, specify, construct, and document the dynamic aspects of a system.
- It shows how the **system behaves and interacts with itself and other entities** (users, other systems).
- They show **how data moves through the system**, how **objects communicate with each other**
- How the passage of **time affects the system**, or what **events cause the system to change internal states**.
- Since behavior diagrams illustrate the behavior of a system, they are used **extensively to describe the functionality of software systems**.

Behavioral Diagram

1. **Use Case Diagram** : Describes the interactions between **users (actors) and the system** through use cases.
2. **Sequence Diagram** : Shows how objects interact in a **specific sequence over time**. It focuses on the order of messages passed between objects during a process.
3. **Activity Diagram** : Represents **workflows or the logic of operations** using activities and decisions. Useful for modeling business processes and complex logic flows.
4. **State Machine Diagram** : Illustrates different **states of an object** and the **transitions** between those states. It's ideal for modeling the behavior of reactive systems or objects with complex lifecycles.

Behavioral Diagram

5. **Communication Diagram** : Displays how **objects interact via messages**, focusing on structural organization. It shows the network of **message-passing** between collaborating objects.

6. **Interaction Overview Diagram** : Combines **activity and interaction diagrams** to give a high-level view of control flow among **interactions**. It simplifies complex scenarios by breaking them into manageable parts.

7. **Timing Diagram** : Depicts changes in **object states or values over time** with precise time constraints. It's useful for real-time and embedded system modeling where timing is critical.

Choosing between a structural and behavioral UML

- Choosing between a structural and behavioral UML diagram depends on what aspect of the system you are trying to understand, design, or explain.

When to Choose a Structural Diagram:

- Use when you're focusing on the static parts of the system, such as:
- Classes and objects and how they relate
- System architecture (components, deployment)

Examples:

- Use Class Diagram to design system structure
- Use Component/Deployment Diagram to plan architecture
- Use Package Diagram for organizing modules

Choosing between a structural and behavioral UML

When to Choose a Behavioral Diagram:

- Use when you're modeling the dynamic behavior or workflow of the system, such as:
- User interactions and system responses
- Object interactions over time
- Process flow and state changes

Examples:

- Use Use Case Diagram to define system functionality
- Use Sequence Diagram to show interaction order
- Use Activity Diagram for workflows
- Use State Diagram to represent object state changes

Simple Rule of Thumb:

- ◆ **What the system is → Use Structural Diagram**
- ◆ **What the system does → Use Behavioral Diagram**

UML Diagrams for Online Shopping Application

Problem Statement:

- Design an **Online Shopping Application** that enables users to **browse products, manage a cart, place orders, and make secure payments**.
- The system should also allow **admins to manage product listings**.
- It must support **user registration, login, order tracking, and handle real-world scenarios** like payment failures or order cancellations.

Objective:

- To model and visualize the **functional structure** and **dynamic behavior** of the online shopping system using **Unified Modeling Language (UML)**.

UML Diagrams for Online Shopping Application

UML Diagrams to be Drawn:

- 1) **Use Case Diagram** : Shows interactions between users (Customer/Admin) and system functionalities.
- 2) **Class Diagram** : Represents the system's static structure including entities like Customer, Product, Order, etc.
- 3) **Sequence Diagram** : Describes the step-by-step interaction (with message flow) between system objects during actions like placing an order.
- 4) **Activity Diagram** : Illustrates the workflow of activities such as user login, product browsing, and order placement.

Online Shopping System– Use Case Diagram

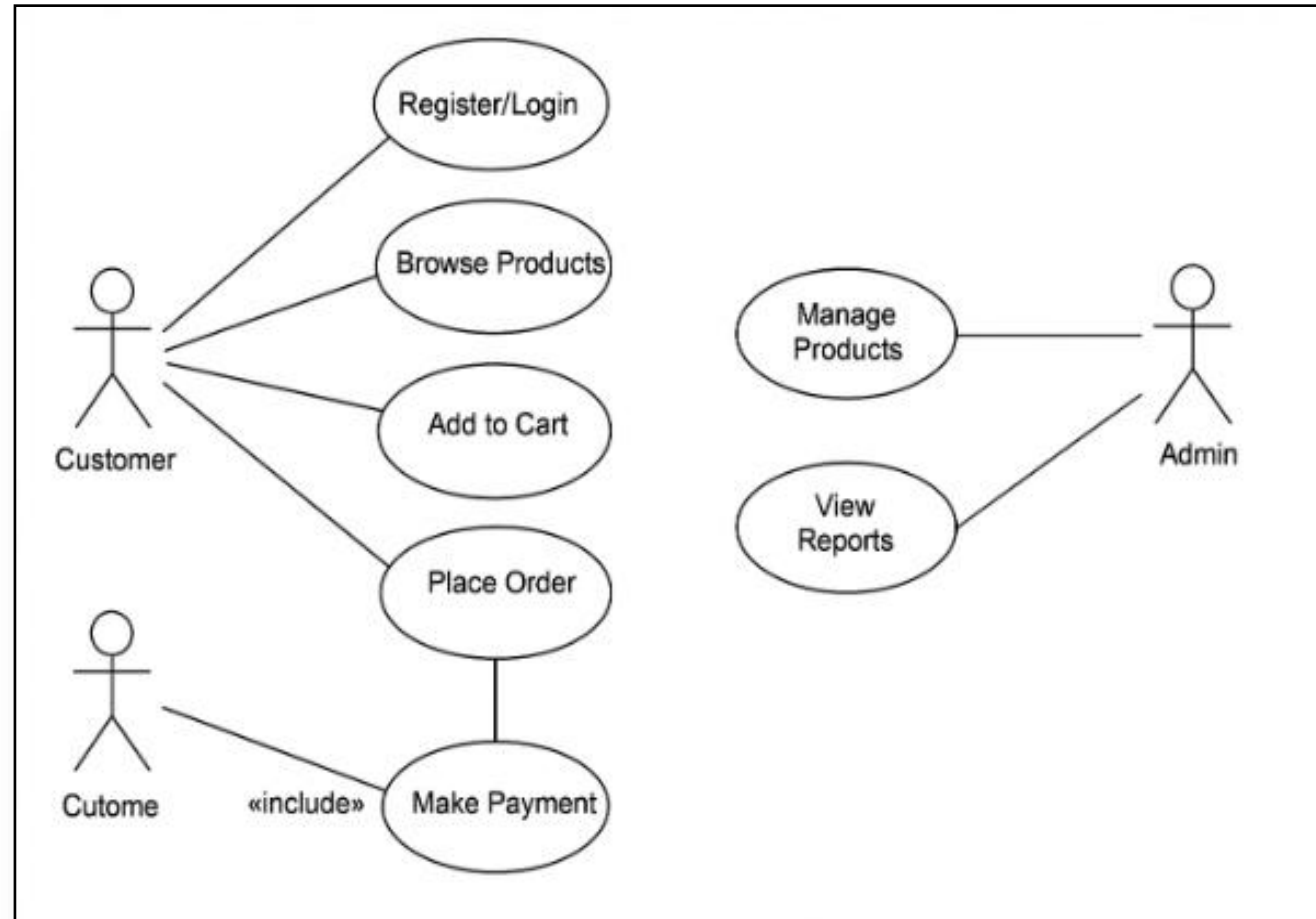
1) Use Case Diagram

Purpose: Shows the interactions between **users (actors)** and the **system**.

Key Use Cases:

- **Customer** can: Register, Login, Browse Products, Add to Cart, Place Order, Track Order.
- **Admin** can: Add/Update Products, Manage Orders.
- **System boundaries** clearly depict what is inside and outside of the system scope.

Online Shopping System– Use Case Diagram



Online Shopping System– Class Diagram

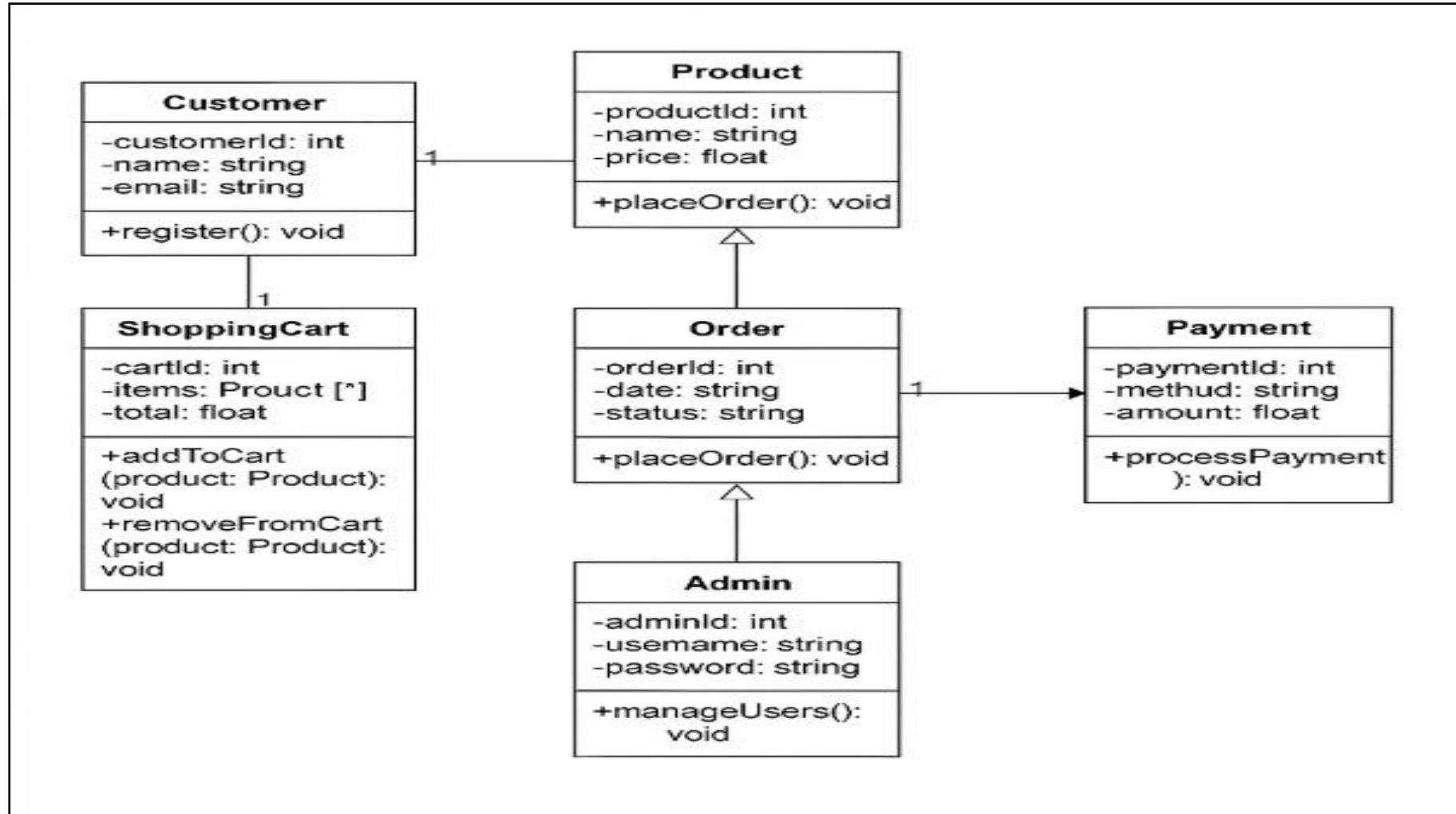
2) Class Diagram

Purpose: Represents the **structure** of the system using **classes**, their **attributes**, **methods**, and **relationships**.

Key Elements:

- **Classes:** Customer, Product, Order, Cart, Payment, Admin.
- **Relationships:**
 - A Customer *places* Orders.
 - An Order *contains* Products.
 - A Cart *holds* Products.
 - Admin *manages* Products and Orders.

Online Shopping System– Class Diagram



Online Shopping System– Sequence Diagram

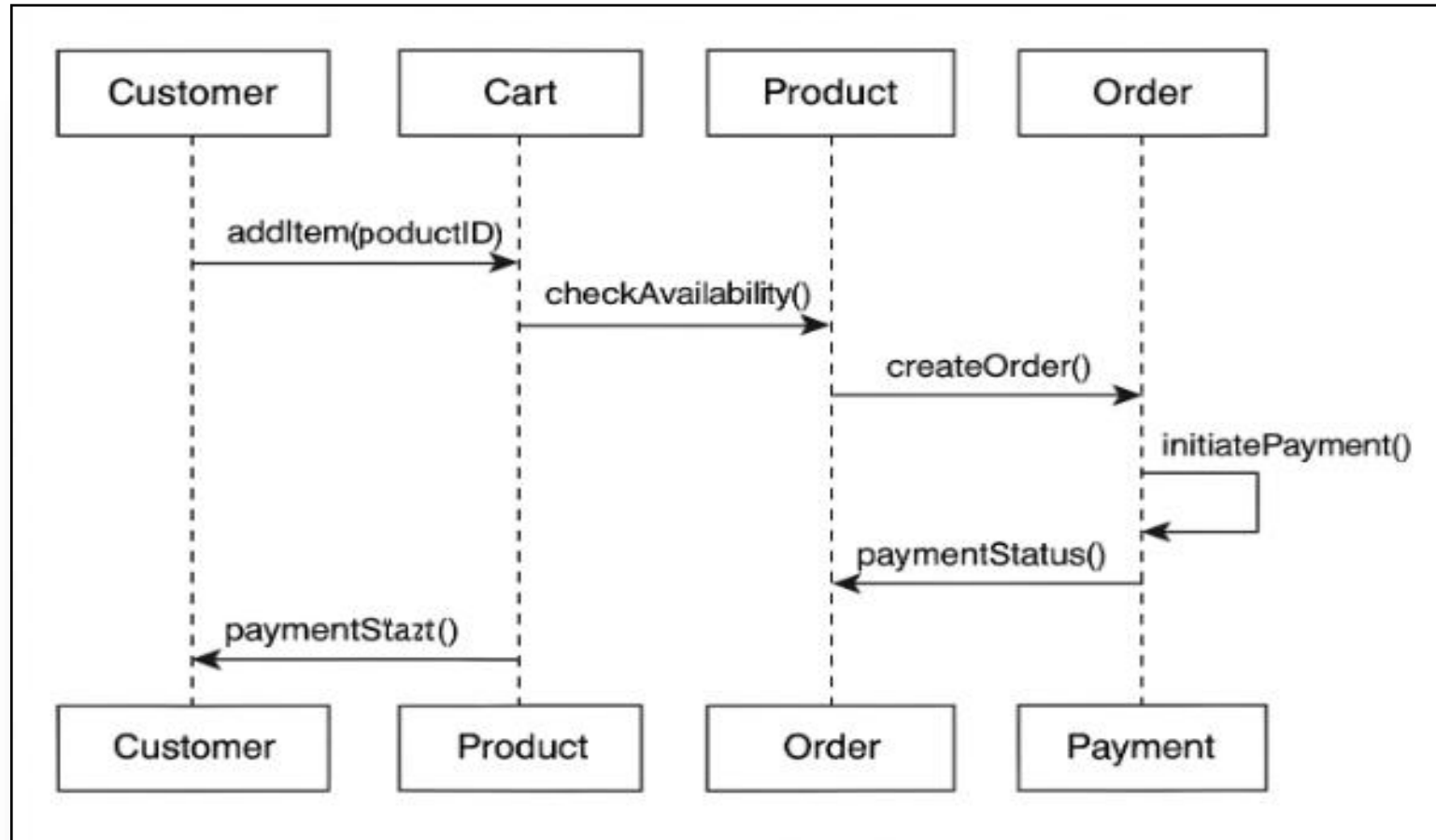
3. Sequence Diagram

Purpose : Describes the **step-by-step interaction** between objects for a specific scenario (**Placing an Order**).

Key Flow:

- Customer logs in.
- Browses and adds items to cart.
- Places an order.
- Payment is processed.
- Order is confirmed and updated in the system.

Online Shopping System – Sequence Diagram



Online Shopping System– Activity Diagram

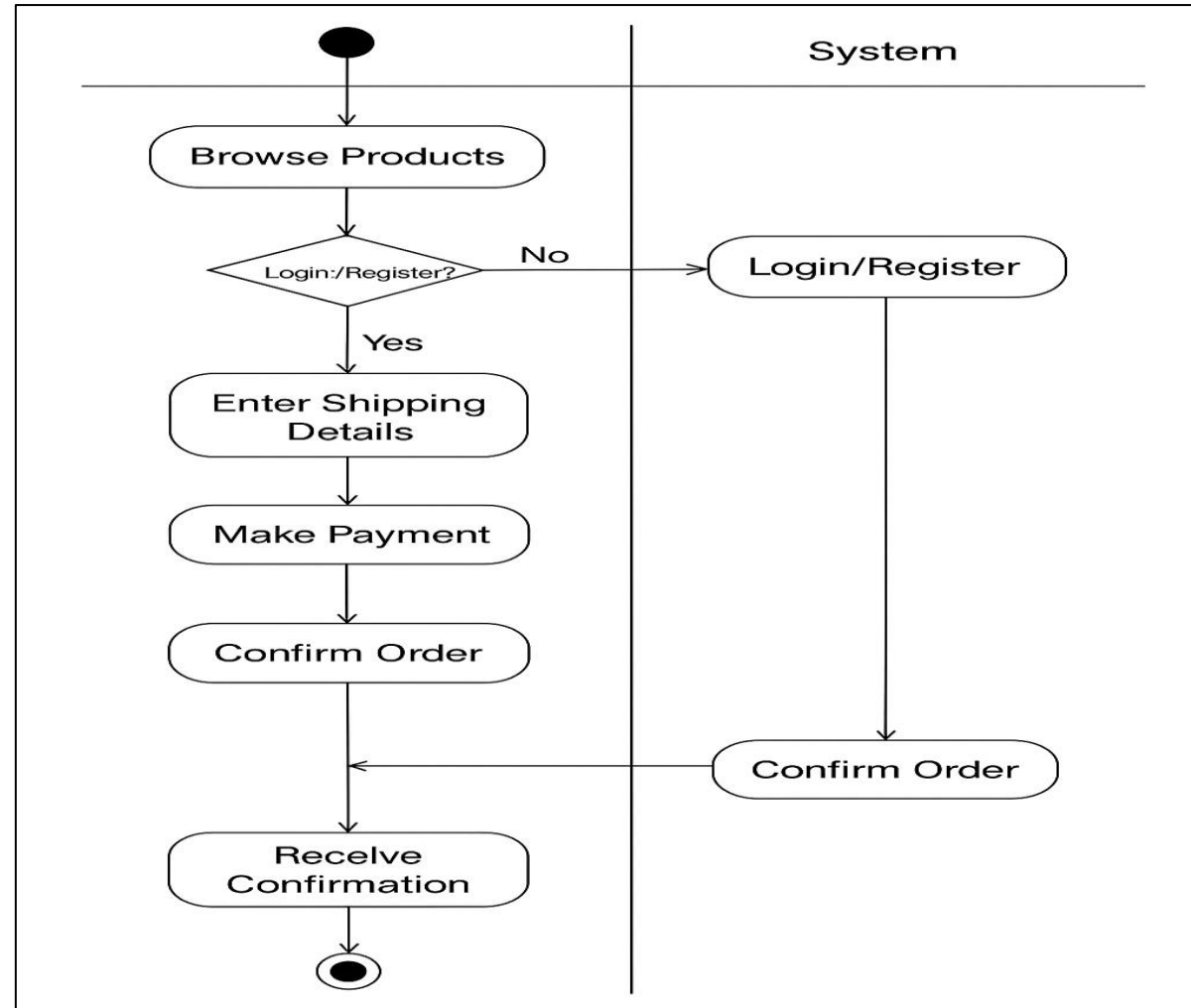
4) Activity Diagram

Purpose: Represents the **workflow of the system** showing activities and decision points.

Key Flow:

- Start → Login/Register → Browse Products → Add to Cart → Checkout → Payment → Confirmation → End.
- Includes **decision points** (e.g., is cart empty?, is payment successful?).

Online Shopping System– Activity Diagram



Software Testing



What is Testing?

- **Software Testing** is the process of **checking whether a software system works correctly** and meets the specified requirements.
- It involves **finding bugs or errors** in the software before it is delivered to users.
- The goal is to ensure the software is **reliable, functional, and free of defects**.
- Testing helps improve the **quality and performance** of the application.

Software Testing

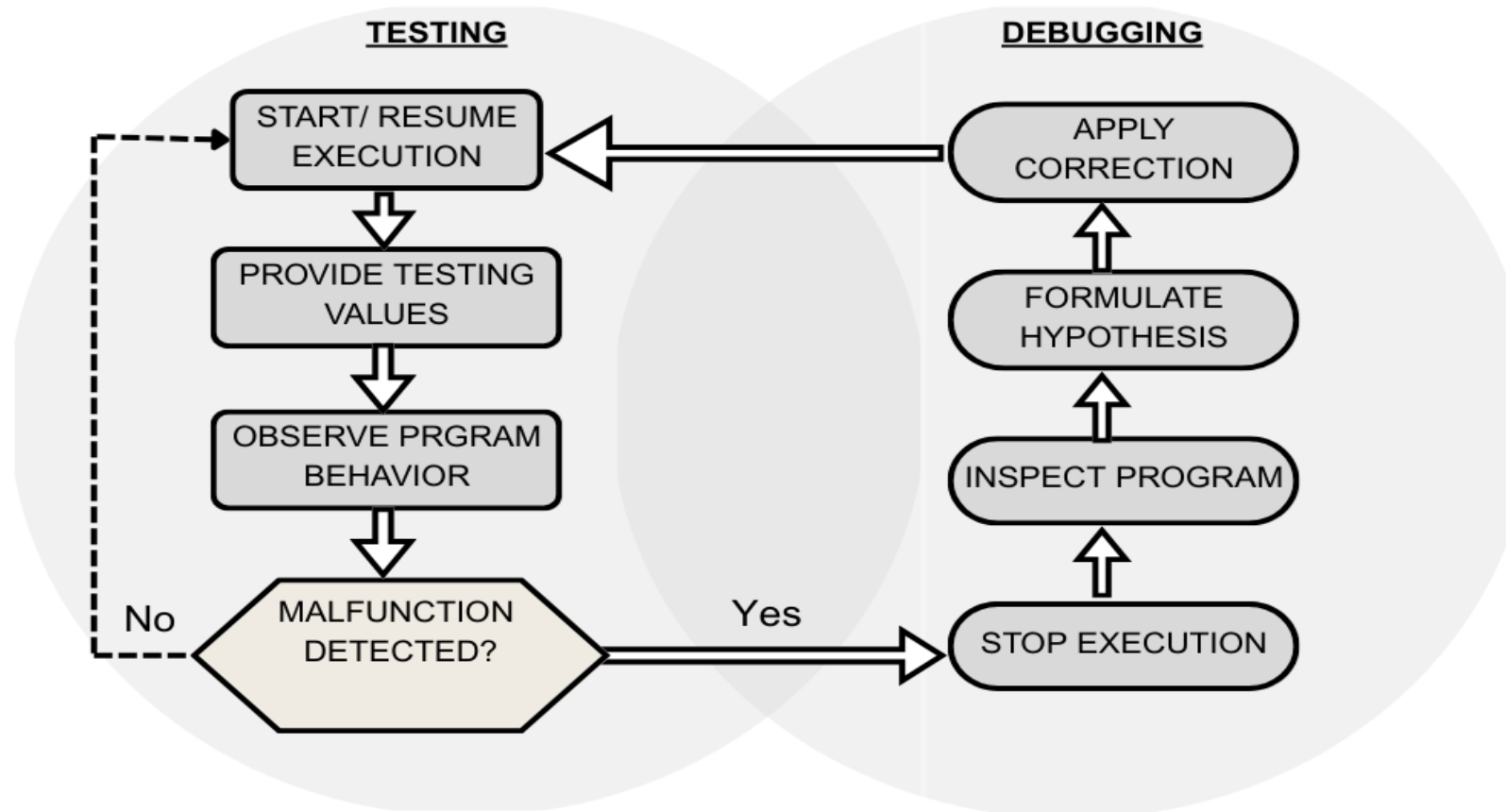
Why is testing necessary ?

- Prevents **loss of money, time, and reputation**
- Avoids **serious failures** like injury or death
- Detects and fixes **defects early**
- Ensures **software quality and reliability**
- Helps meet **legal, contractual, and industry standards**
- Supports **safe updates and maintenance**
- Improves **user satisfaction and trust**

Testing vs Debugging

Aspect	Testing	Debugging
What it is	Process of finding defects by executing code	Process of analyzing and fixing those defects
Who does it	Usually done by testers	Usually done by developers
Goal	To identify failures in the software	To locate and fix the root cause of the failure
When it happens	After development or during updates	After a failure is found during testing
Involves	Running test cases, checking outputs	Code analysis, changes, and re-testing

Testing vs Debugging



Software Testing

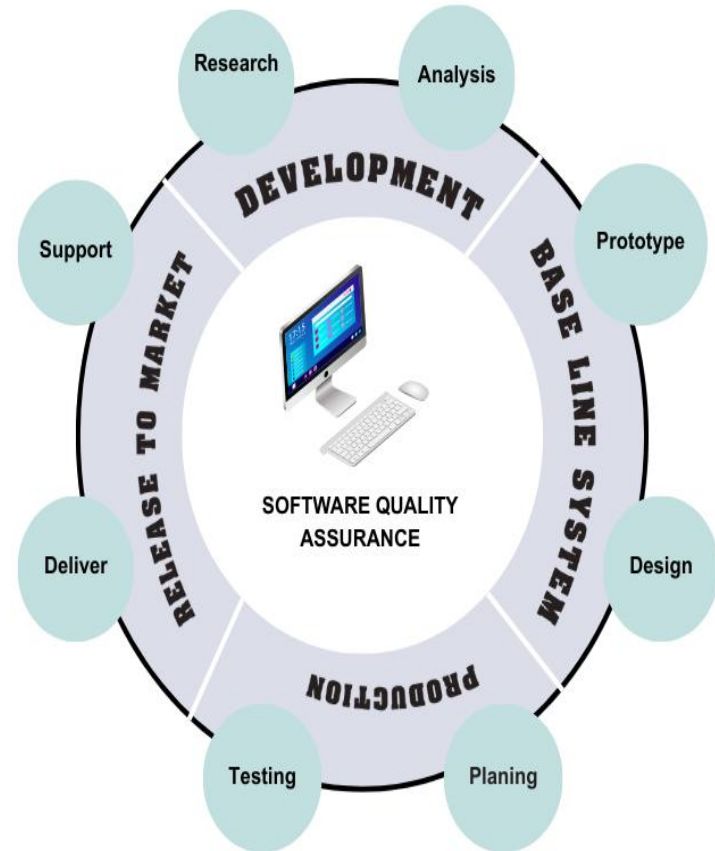
Errors, Defects, and Failure

Term	Definition	Cause	Detected By	Example
Error	A human mistake made by a developer during coding, design, or requirements	Human mistake	Developer	A developer misuses a variable or writes incorrect logic in a function
Defect	A flaw or bug in the software code caused by an error	Programming or design error	Tester during testing	A login button doesn't work due to incorrect event handling
Failure	The software behaves unexpectedly or doesn't meet requirements during execution	One or more defects reaching execution	End-user or during testing	App crashes when submitting a form or incorrect output is shown

Software Testing

Quality Assurance

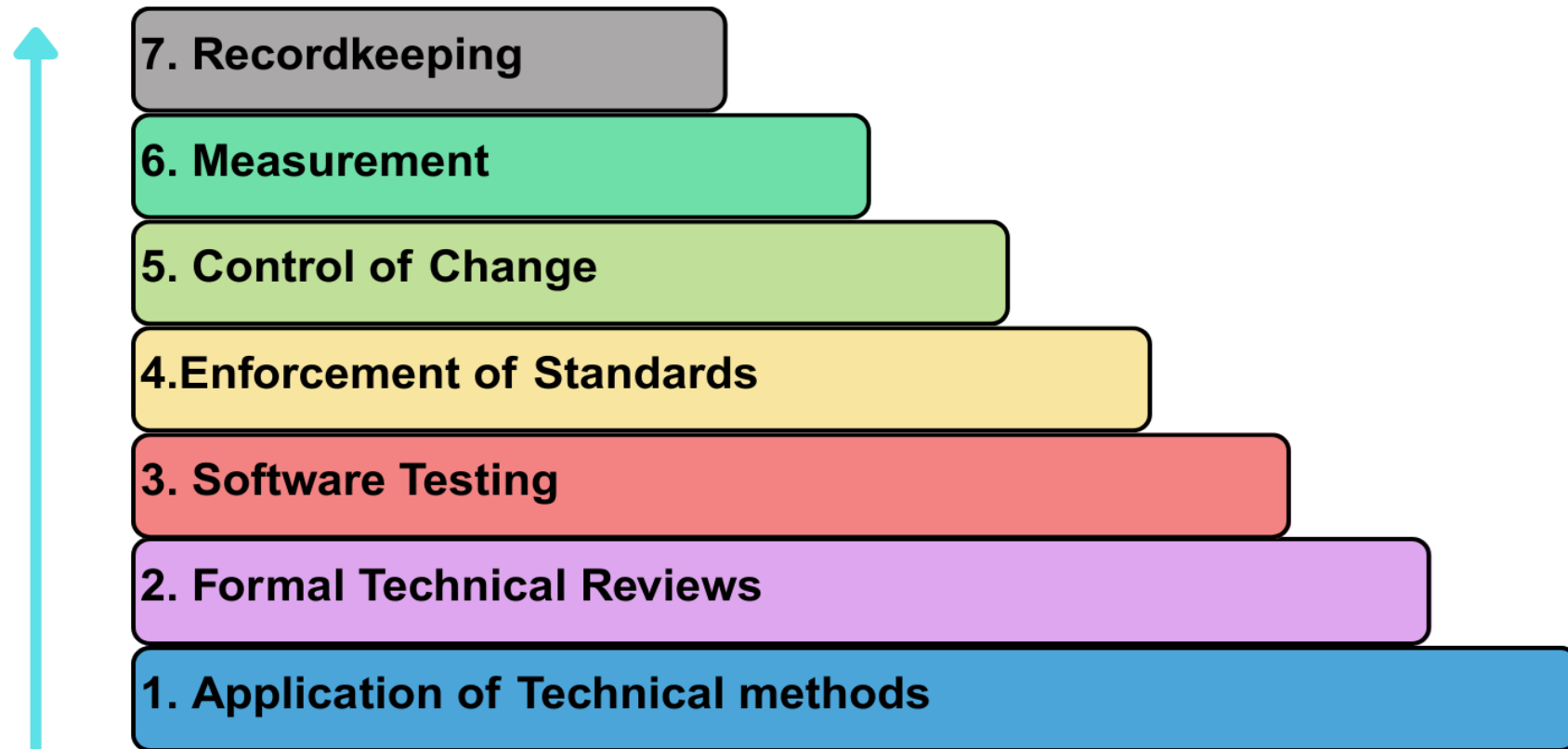
- **Quality Assurance (QA)** is the process of ensuring that software meets defined standards and works as expected.
- It focuses on **preventing defects** by following proper development processes, guidelines, and testing methods to deliver **high-quality, reliable software**.



Key activities of Quality Assurance

1. **Application of Technical Methods** : Using structured methods like design modeling and coding best practices to reduce errors.
2. **Formal Technical Reviews** : Conducting peer reviews to detect defects and ensure compliance with standards.
3. **Software Testing** : Executing the software to find and fix bugs before release.
4. **Enforcement of Standards** : Making sure all team members follow defined coding and process standards.
5. **Control of Changes** : Managing updates carefully to avoid introducing new defects.
6. **Measurement** : Collecting data to monitor and improve software quality and process performance.
7. **Record Keeping** : Documenting all activities and changes for tracking, analysis, and future reference.

Key activities of Quality Assurance

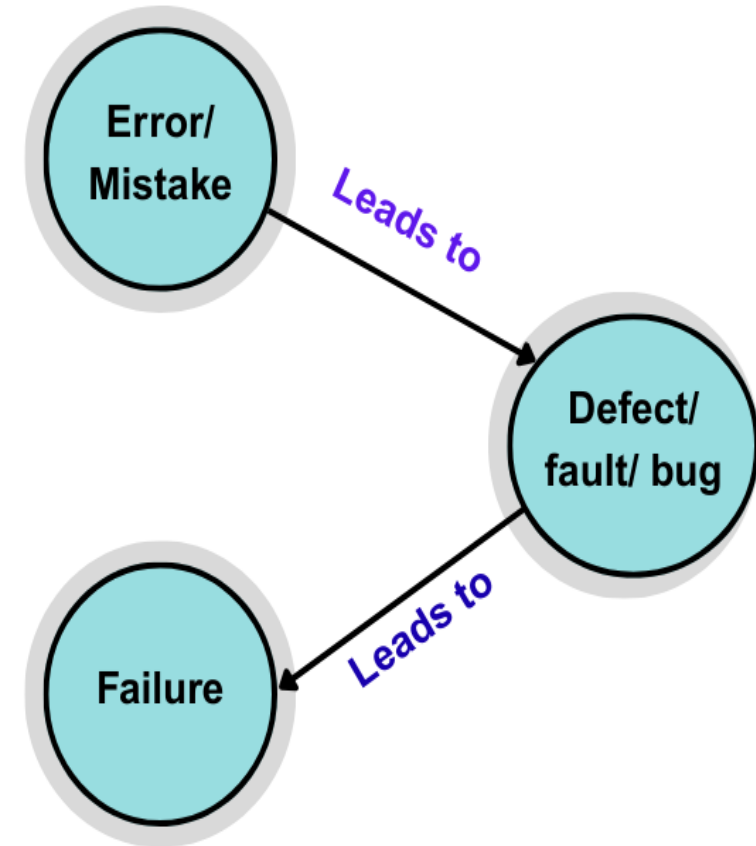


Software Quality Assurance

Quality Control

Errors may occur for many reasons, such as:

- **Time** pressure
- **Human** fallibility
- **Inexperienced** or **insufficiently** skilled project participants
- **Miscommunication** between project participants, including miscommunication about requirements and design



Difference between Quality Assurance and Quality Control

Aspect	Quality Assurance (QA)	Quality Control (QC)
Focus	Prevention of defects	Detection of defects
Goal	Ensure proper processes are followed	Ensure final product meets quality standards
When Performed	Throughout the development process	After the product is developed or during testing
Activities	Process audits, standards enforcement, reviews	Testing, inspections, identifying bugs
Responsibility	Done by everyone involved in the process	Done by testing or QA teams

Software Testing

Test Activities and Tasks

- Test planning
- Test monitoring Control
- Test analysis
- Test design
- Test implementation
- Test execution
- Test completion/closure

Test Planning	• Actor: Test Lead, Test manager, Project lead, Project Manager. • Out Put: Test Policy, Test Strategy, Team Plan
Test Analysis	• Actor: Test Engineer • Output: Requirements Clarification Note (RCN)
Test Design	• Actor: Test Engineer • Output: Test Scenarios, Test Cases, Test Data, RTM [2]
Test Execution	• Actor: Test Engineer • Output: Execution of Test Cases, Identifying and Reporting Defects, Re Test [2]
Test Closure	• Actor: Project Lead, Project Manager, Test Lead, Test Manager • Output: Test Summary Reports [2]

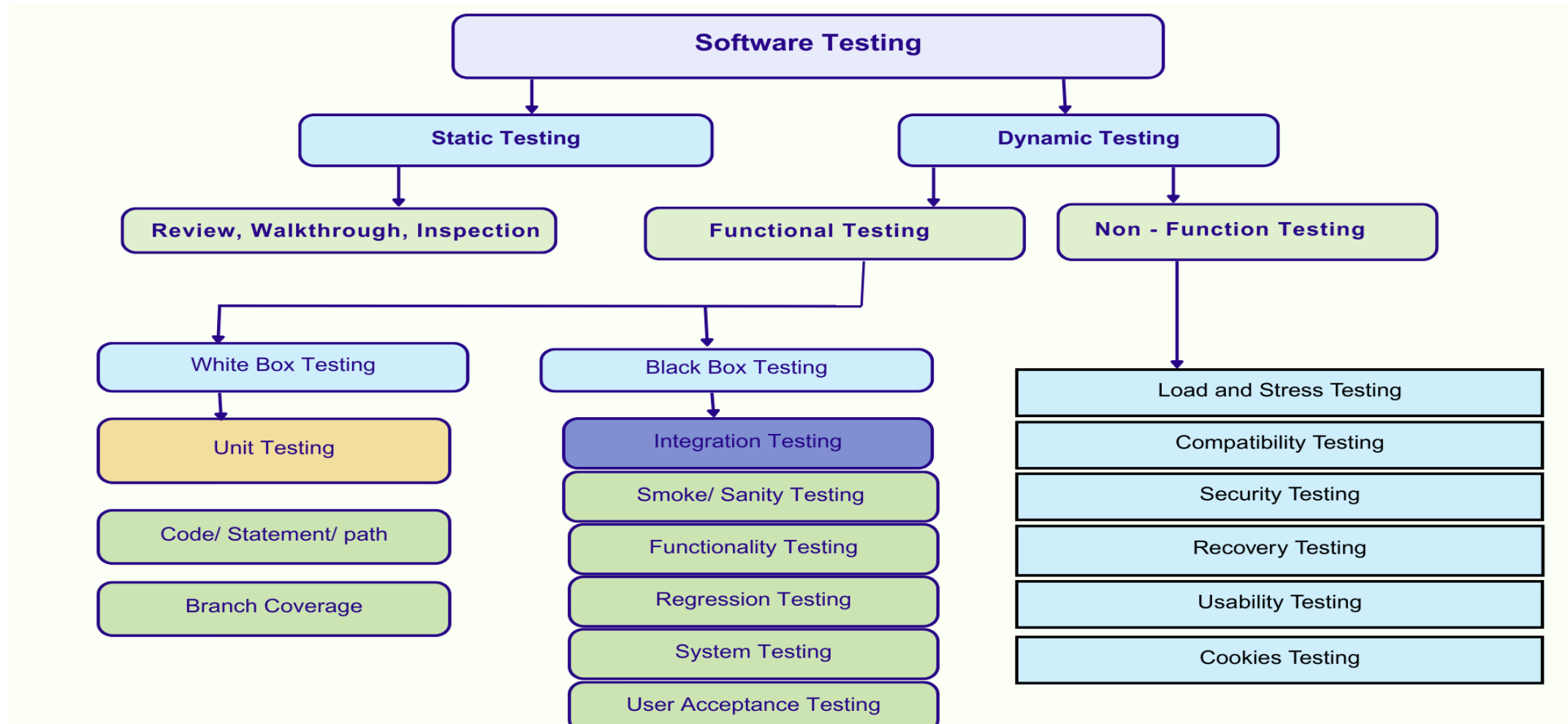
Types of Software Testing

Software testing is broadly classified into **static and dynamic testing**, based on what aspect of the software is being tested.

- **Static Testing:** Involves reviewing the **software work products** (like code, documents, or requirements) without executing the code. It helps find **errors early** through techniques such as **inspections, walkthroughs, and code reviews**.
- **Dynamic Testing:** Involves executing the actual software to **validate its behavior and functionality**. It helps **identify defects during runtime** by checking how the system responds to inputs.

Software Testing

Types of Software Testing



Types of Software Testing

Dynamic testing is further classified into Functional and Non-Functional testing, based on **what aspect of the software is being tested.**

- **Functional Testing** focuses on verifying **what the system does**. It checks the system's behavior against the functional requirements, such as login, user registration, or form submission.
- **Non-Functional Testing** checks how the **system performs under certain conditions**. It includes aspects like performance, security, usability, and reliability.

Types of Software Testing

- **Black Box Testing** : Functional Testing approach where internal code is not visible.
 - It includes most user-level tests like system, regression, and UAT
- **White Box Testing** : Code-based Testing where internal logic and structure are fully tested.
 - Mainly Non-Functional Testing (but used in Unit Testing, which supports functional testing).It includes unit testing, code path testing, and loop testing — mostly done by developers.
- **Grey Box Testing**: Hybrid Testing with partial knowledge of internal code and design.
 - Used in integration, security, and web application testing where testers know some internals.

Types of Software Testing

These are the main types of functional testing:

- **Unit testing:** a type of testing done on an individual unit in isolation from the rest of the application. A unit is the smallest testable part of any software, usually responsible for only a very targeted set of functionalities.
- **Integration testing:** a type of testing where individual units or components are combined and tested as a group to ensure they work together as expected.
- **System testing:** covers the end-to-end functions of a system, and thus it provides reliability to the system.
- **Acceptance testing:** a type of testing to evaluate the application against real-life scenarios

Types of Software Testing

- Under Non-functional Testing, there are also many common testing types, each with different objectives and strategies:
- **Security Testing:** Testing that checks if the software is secure and protects against unauthorized access or threats.
- **Performance Testing:** Testing that assesses how well the software performs in terms of speed, stability, and resource usage.
- **Load Testing:** A type of performance testing that evaluates how the software handles expected and peak loads.

Types of Software Testing

- **Usability Testing:** Testing that measures how user-friendly and easy-to-use the software is.
- **Compatibility Testing (or Cross-browser Testing):** Testing that ensures the software works correctly across different platforms, devices, or environments.
- The decision to use which of these types of software tests depends on the test scenarios, resource availability, and business requirements.

Functional Versus Non-Functional Testing

Testing aspects	Functional testing	Non-functional testing
Involves	Product features and functionality	Quality factors
Tests	Product behavior	Behavior and experience
Result conclusion	Simple steps written to check expected results	Huge data collected and analyzed
Results varies due to	Product implementation	Product implementation, resources, and configurations
Testing focus	Defect detection	Qualification of product
Knowledge required	Product and domain	Product, domain, design, architecture, statistical skills
Failures normally due to	Code	Architecture, design, and code
Test case repeatability	Repeated many times	Repeated only in case of failures and for different configurations
Configuration	One-time setup for a set of test cases	Configuration changes for each test case

Software testing metrics

- In software testing, key metrics help **evaluate** the **quality, complexity, and maintainability** of the code.
- These metrics provide quantitative insights that guide **better design, improve testability, and support decision-making.**
- Commonly used **metrics** like Lines of Code (LOC), Cyclomatic Complexity, Coupling, and Cohesion help engineers measure how **efficient and manageable the software is.**
- Understanding these metrics is **crucial** for both **development** and **quality assurance teams.**

Key Metrics for Evaluation

Measure software quality, complexity, and maintainability.

- **LOC (Lines of Code):** Measures the size of software by counting lines written; useful for productivity & effort estimation.
- **Cyclomatic Complexity:** Indicates code complexity by counting independent paths; higher value = more complex & harder to test.
- **Coupling:** Measures how strongly modules depend on each other; low coupling is preferred for maintainability.
- **Cohesion:** Measures how closely related functions within a module are; high cohesion indicates better design.

THANK YOU